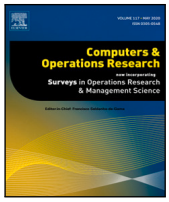
Computers & Operations Research 172 (2024) 106791



Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor





A Graph Reinforcement Learning Framework for Neural Adaptive Large Neighbourhood Search

Syu-Ning Johnn [a,*,1], Victor-Alexandru Darvariu [b,1], Julia Handl [c], Jörg Kalcsics [a]

[a] School of Mathematics, University of Edinburgh, Edinburgh, United Kingdom
[b] Department of Computer Science, University College London, London, United Kingdom
[c] Alliance Manchester Business School, University of Manchester, Manchester, United Kingdom

ARTICLE INFO

Keywords:
Machine Learning
Adaptive Large Neighbourhood Search
Markov Decision Process
Deep Reinforcement Learning
Graph Neural Networks

ABSTRACT

Adaptive Large Neighbourhood Search (ALNS) is a popular metaheuristic with renowned efficiency in solving combinatorial optimisation problems. However, despite 18 years of intensive research into ALNS, the design of an effective adaptive layer for selecting operators to improve the solution remains an open question. In this work, we isolate this problem by formulating it as a Markov Decision Process, in which an agent is rewarded proportionally to the improvement of the incumbent. We propose Graph Reinforcement Learning for Operator Selection (GRLOS), a method based on Deep Reinforcement Learning and Graph Neural Networks, as well as Learned Roulette Wheel (LRW), a lightweight approach inspired by the classic Roulette Wheel adaptive layer. The methods, which are broadly applicable to optimisation problems that can be represented as graphs, are comprehensively evaluated on 5 routing problems using a large portfolio of 28 destroy and 7 repair operators. Results show that both GRLOS and LRW outperform the classic selection mechanism in ALNS, owing to the operator choices being learned in a prior training phase. GRLOS is also shown to consistently achieve better performance than a recent Deep Reinforcement Learning method due to its substantially more flexible state representation. The evaluation further examines the impact of the operator budget and type of initial solution, and is applied to problem instances with up to 1000 customers. The findings arising from our extensive benchmarking bear relevance to the wider literature of hybrid methods combining metaheuristics and machine learning.

1. Introduction

The Adaptive Large Neighbourhood Search (ALNS) metaheuristic was first introduced by Ropke and Pisinger (2006) to solve Combinatorial Optimisation Problems (COPs). It iteratively deconstructs and reconstructs parts of the solution in order explore unvisited parts of the search space towards promising directions. This "relax-and-reoptimise" perturbation process is based on sequentially executing a pair of destroy and repair heuristic algorithms called *operators*, each of which specifies a way of mapping one solution to another solution in its neighbourhood. It typically also features a solution acceptance mechanism, such as Simulated Annealing, which helps to escape local minima.

A natural question that arises is how to best select operators such that the solution is improved. After 18 years of intensive research into ALNS, this nevertheless remains an open question. An ubiquitous method is to use an adaptive Roulette Wheel (RW) mechanism (Mara et al., 2022), which maintains a set of weights corresponding to each operator that are used to derive selection probabilities. Weights are learned in an online fashion based on the quality of the solutions obtained when applying the operators. Despite the pervasiveness of RW, a recent meta-review by Turkeš et al. (2021) has found that it achieves a mere 0.14% improvement in solution quality on average, with several works opting for omitting it altogether from the ALNS framework (Grangier et al., 2017; Lehuédé et al., 2014; Tellez et al., 2018). Clearly, a better mechanism is required.

Beyond the optimality of choosing the right operator out of a predetermined set when executing the search, one must also consider the choice of which operators to include in the portfolio to begin with. This is a process that requires a significant human and computational cost for experimentation to determine effective operators. Additionally, while selecting operators according to their individual performance may seem reasonable, this fails to account for interactions arising when multiple ones are applied in combination (Kallestad et al., 2023). Finally, the choice of the operator portfolio size is a delicate one. Too small a portfolio may lead to ineffective exploration of the search space

* Correspondence to: Department of Chemical Engineering, University College London, United Kingdom.
E-mail address: syu.johnn@gmail.com (S.-N. Johnn).
[1] Equal contribution and joint first authorship.

https://doi.org/10.1016/j.cor.2024.106791
Received 16 February 2024; Received in revised form 6 June 2024; Accepted 31 July 2024
Available online 2 August 2024
0305-0548/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

due to the diversity of the solutions being limited; while too large a portfolio can prove challenging for the operator selection mechanism due to the increased number of possible choices.

In the literature, the primary foci of researchers have been comparing the speed and solution quality of ALNS to those obtained by another exact method or heuristic, as well as applying ALNS to new problem variants. Comparatively less attention has been dedicated to the performance of ALNS components and how to improve them individually. In contrast, *the core goal of our work is to design an effective yet principled operator selection mechanism*.

We achieve this through the use of Machine Learning (ML), an increasingly important tool for combinatorial optimisation (Bengio et al., 2021). Concretely, we opt for Reinforcement Learning (RL), a framework for learning how to optimise a given reward function by trial-and-error. In the ALNS context, an agent is rewarded for selecting sequences of operators that lead to the largest improvements in the objective function value of the solution. Compared to supervised learning, which requires labelled examples, RL is able to *discover* optimal operator sequences in a data-driven way. This may be seen as automating the costly human process of designing operator portfolios and selecting operators appropriate at different stages of the search.

More specifically, in this work, we make the following contributions:

- We formulate the choice of a sequence of destroy and repair operators as a standalone Markov Decision Process (MDP), in which an agent receives a reward proportional to the improvement in a solution that can be captured as a graph.
- We propose two methods for learning to select operators. The first is based on Deep RL and uses a Graph Neural Network (GNN) for state representation, which allows for end-to-end learning of rich features while generalising to problem instances of different sizes. The second is a lightweight method that is inspired by the classic RW update, and features weights that are learned in a prior training phase, rather than fully online.
- We carry out an extensive evaluation on 5 routing problems with a representative selection of operators from the literature. The approaches are successfully compared to a wide selection of baselines. Practical aspects such as the budget of operators afforded to ALNS, means of constructing initial solutions, and scalability to large instances are considered.
- Our evaluation shows that both proposed methods outperform the classic RW and uniform random operator selection. Our Deep RL method is also shown to outperform those proposed in concurrent work by Kallestad et al. (2023) and Reijnen et al. (2024) in 90.4% of cases. These aspects support the contribution of our work to the ALNS literature, as well as to that of hybrid methods at the intersection of metaheuristics and ML more broadly.

The present work evolved out of a paper presented at the OLA 2023 conference (Johnn et al., 2023). Building on this initial effort, we have generalised the methodology to 5 routing problems, and substantially expanded the considered operator portfolio from 12 destroy and 2 repair operators to 28 and 7 operators respectively. Furthermore, we have implemented and integrated an RL method based on concurrent work (Kallestad et al., 2023; Reijnen et al., 2024), to which our proposals are compared. We also carried out an evaluation on very large instances with up to 1000 customers, as well as different initial solutions. Finally, we have expanded the literature review as well as the level of detail throughout the manuscript.

The rest of this paper is organised as follows. In Section 2, we review related work in ALNS and ML for combinatorial optimisation. In Section 3, we propose our methodology for combining the ALNS framework with RL towards a principled and more performant operator selection mechanism. The experimental setup and a computational study are presented in Sections 4 and 5, respectively. Finally, a conclusion and directions for future research are given in Section 6.

2. Literature review

In this section, we first review important works in the ALNS literature and offer a critical perspective on its current status. We then review the heuristic operators for routing problems that have been proposed and identify their commonalities. Subsequently, we discuss existing studies in ML for combinatorial optimisation, especially those that combine ML and metaheuristics or use a GNN representation. Finally, we dedicate a subsection to a detailed comparison between our work and other concurrent efforts to improve ALNS using RL.

2.1. Adaptive large neighbourhood search

ALNS belongs to the class of metaheuristics, a group of problem-independent optimisation guidelines applicable to a broad range of problems. Based on Shaw's principle of Large Neighbourhood Search (LNS) (Shaw, 1999), ALNS is equipped with two additional mechanisms that determine the performance of the perturbation process. Firstly, unlike Shaw's LNS, which only features a single pair of destroy and reinsertion algorithms, ALNS contains multiple predefined operators and an adaptive layer that iteratively selects and applies them based on their past success rates. This adaptive layer involves an operator selection mechanism, typically an embedded RW algorithm (Mara et al., 2022). The second distinction lies in the solution acceptance mechanism. Conventional options such as Simulated Annealing (SA), record-to-record travel, and threshold acceptance have been commonly employed in the ALNS framework (Santini et al., 2018) to reduce the chance of the search process becoming trapped prematurely at local optima. This is an improvement from the classical LNS framework, which uses a greedy acceptance criterion that admits only strictly-improving solutions and, as a result, helps to balance the exploration of the unvisited parts and exploitation of the locally optimal parts of the solution space.

The ALNS metaheuristic is renowned for its effectiveness in generating good-quality solutions within reasonable computational time. However, despite its extensive application for solving a variety of COPs, the specific contributions of each ALNS component towards its general performance are not well-explored or understood. In a recent state-of-the-art review of ALNS (Mara et al., 2022), it was highlighted that only a meager 2 out of 252 papers have gone beyond the straightforward implementation and delved into in-depth component-based analysis. Notably, Santini et al. (2018) focused on the selection of the ALNS acceptance criterion, and Turkeš et al. (2021) investigated the effectiveness of the ALNS adaptive layer for operator selection. These studies, while shedding light on key aspects of ALNS, nevertheless indicate the need for further research and in-depth analysis of its individual components, so that we may enhance its overall performance and advance our understanding of the metaheuristic.

Let us summarise the three main deficiencies that exist in the current ALNS framework. Firstly, several studies have indicated that the ALNS adaptive layer has limited capability to dynamically select the best operators, despite being intentionally engineered for this purpose. The recent meta-analysis on ALNS by Turkeš et al. (2021) reported a mere 0.14% average improvement brought by the adaptive layer from the analysis of 25 ALNS implementations in the literature, emphasising the need for a more efficient operator selection mechanism. Guided by preliminary experiments, a few authors including Lehuédé et al. (2014), Grangier et al. (2017) and Tellez et al. (2018) went as far as to replace the adaptive layer with a uniform random choice or to omit it altogether. Moreover, the frequent attempt of hybridisation with a further local search procedure indicates that the incumbent created by the selected operator pairs may not have sufficiently good performance after all (Mara et al., 2022). This poor adaptability may arise from certain groups of operators being more effective during specific stages of the search process and not useful during another, which can fail to be detected by an adaptive layer that relies on periodic updates

of selection probabilities. Consequently, if the operator selector is not carefully fine-tuned, it can easily result in suboptimal performance.

Secondly, operator portfolio design can require considerable manual evaluation (Mara et al., 2022) and remains largely handcrafted based on expert knowledge, as there is no clear problem-oriented guideline of which configuration provides the best objective value improvement for a specific problem. Naturally, the process of identifying an efficient set of operators from the existing literature entails a significant amount of unavoidable manual testing, which is compounded by the burden of an insufficient adaptive layer. It also necessitates manual efforts in designing problem-specific operators as well as adapting existing ones to fit the problem features. These represent common drawbacks of classic heuristic design (e.g., see Khalil et al. (2017), Wu et al. (2021b)). and limits the adaptability of the ALNS metaheuristic to generalised problems.

Evaluating an operator's efficiency by its average performance can be deceptive. As suggested by Kallestad et al. (2023), while handpicking individual operators based on their average performance may seem reasonable, it can result in neglecting good performance at specific moments on the search trajectory, thus hindering the overall power of an operator set. In addition, the complexity of operator selection increases when the integrated performance of an operator set surpasses the individual performance of its components. Some combinations of operators may also have a detrimental impact on the overall performance. Therefore, sequentially including or eliminating operators until attaining a certain set may lead to a suboptimal outcome. Finding the optimal combination of operators that gives the best solution will result in a subset selection problem that is NP-hard. Alternatively, we require a costly factorial analysis to systematically explore the best integrated performance out of all operator combinations for any particular problem.

Lastly, the choice of portfolio size is also delicate: too few operators might not enable the search to visit unexplored neighbourhoods, but a plethora of operators can introduce noise to the adaptive layer, especially when some of the operators may not be helpful. Thomas and Schaus (2018) proposed a blackbox ALNS equipped with a portfolio of 30 destroy and 36 repair operators grouped into several categories to fit specific neighbourhood structures. Despite achieving the best performance out of 25 ALNS implementations in the meta-analysis by Turkeš et al. (2021), the authors showed that only a handful of operators are particularly beneficial for a given COP. They even indicated that the non-adaptive version of ALNS with an appropriately selected operator pair can outperform ALNS with many. Tellez et al. (2018) also indicated that a few selected operators are sufficient to achieve a good solution, while including any additional ones may lead only to a marginal improvement. As a result, the task of refining an appropriate portfolio as well as enhancing the accuracy with which the best possible operator at a given time is selected have become increasingly crucial in the pursuit of better heuristic performance, especially when tackling a broader spectrum of optimisation problems characterised by numerous problem-specific neighbourhoods.

2.2. ALNS operator classification

As previously mentioned, an ALNS iteration consists of alternating between applying a destroy operator (which removes a certain number of nodes from the solution and temporarily places them inside a pool) and a repair operator (which re-inserts the nodes back into the incomplete solution). In the literature, operators are carefully tailored to fit different problem structures and features. Despite the large variety of operator designs, the mechanisms behind them are surprisingly similar to the first version of ALNS proposed by Ropke and Pisinger (2006). We conducted a thorough analysis of operators in the literature and have identified the following 3 classes: *random-based* operators randomly remove or insert a set of nodes to explore unvisited parts of the search space; *greedy-based* operators (for both the destroy and repair phases) remove or insert the top-ranking set of nodes with the highest proximity with respect to specific measures in order to exploit local structure; and the *related-based* operators are a hybrid of the two, balancing diversification and intensification. Thus, if we select representative operators from each class, we can apply fewer operators to achieve the same perturbation goal. For brevity, we enumerate the operators in the literature in Table B.7 together with relevant references and descriptions. Section 4.2 discusses further details that are relevant to the performed evaluation.

2.3. Machine learning for combinatorial optimisation

In recent years, the use of ML techniques for solving highly complex COPs has witnessed a resurgence, following pioneering work in the 1980s and 1990s (Hopfield and Tank, 1985; Smith, 1999). ML may be used in several different manners in solver pipelines (Bengio et al., 2021), including constructing the solution from scratch using learned knowledge, and alongside classic heuristic and exact algorithms that query the ML model during the optimisation process. While routing problems such as the Vehicle Routing Problem (VRP) and its variants (Bai et al., 2023) have received particular attention, ML is proving to be a promising component of optimisation pipelines in general.

A particularly effective use of ML models to directly construct solutions is via RL, a trial-and-error paradigm for producing goal-directed behaviour. RL enables learning heuristic algorithms for addressing COPs without designing them manually by optimising the gained rewards. As such, following a training phase, RL may lead to the implicit discovery of better-performing heuristics than those that are currently known.

Several pioneering studies applied RL to directly construct solutions for routing-related problems. Bello et al. (2016) were the first to apply policy gradient algorithms to tackle the Travelling Salesperson Problem (TSP). They developed a neural combinatorial optimisation framework and trained it with end-to-end RL to learn approximate solutions using the tour length as a reward signal. Khalil et al. (2017) addressed three graph optimisation problems including the minimum vertex cover, maximum cut and TSP. They used Q-learning to train a greedy policy parameterised by a GNN to incrementally construct solutions, with the set of actions determined by the solution state. Nazari et al. (2018) proposed an end-to-end framework that outputs solutions directly from the routing-based problem instances. Their method can efficiently solve various routing problem instances of similar size, and outperforms many classic VRP heuristics. Kool et al. (2018) proposed a construction heuristic that consists of an attention-based decoder trained with RL to autoregressively build solutions for the TSP and its variants. Kim et al. (2021) proposed a collaborative scheme combining two iterative Deep RL policies that construct and improve candidate solutions to various routing problems. Other variants of using RL to tackle routing problems can be found in the work of Deudon et al. (2018), Qiu et al. (2022) and Zhang et al. (2020).

ML techniques can also be applied to enhance the performance of heuristics and metaheuristics (Talbi, 2021). The reader is referred to Karimi-Mamaghan et al. (2022) for a comprehensive review. Many studies applying RL with existing metaheuristics, including local search (Zhou et al., 2016), simulated annealing (Mosadegh et al., 2020), variable neighbourhood search (Kalatzantonakis et al., 2023), memetic search (Handoko et al., 2014), genetic algorithms (Ahmadi et al., 2018), artificial bee colony algorithms (Durgut et al., 2021), and other optimisation processes (Zhao et al., 2021). Other popular research avenues explore hyperheuristics as a search paradigm for selecting among heuristics or designing them automatically (Burke et al., 2013), featured in recent works through its application with ML (Lagos and Pereira, 2024; Zhang et al., 2022), and automatic algorithm configurations for finding the most suitable parameter setting of an optimiser (López-Ibáñez et al., 2016).

Traditional improvement heuristics are typically handcrafted algorithms that require extensive domain knowledge and a considerable investment of effort and time. Yet, despite being carefully handcrafted by domain experts, classic heuristics often struggle to achieve satisfactory performance. This is due to both the intrinsic difficulties of the problems they tackle as well as the need to adapt them when problems change even slightly. Often, designing afresh for each new problem and numerous trial-and-error experiments are unavoidable before securing good heuristic performance (Khalil et al., 2017). To overcome these limitations, incorporating ML techniques within the classic solution framework offers a viable alternative to alleviate the complexity associated with manually designing algorithms. Lu et al. (2019) presented a learn-to-improve framework for VRP to iteratively refine an initial solution with customised improvement operators chosen by an RL-based controller. Chen and Tian (2019) proposed a Deep RL-based algorithm to iteratively select and rewrite the local components of an existing solution until convergence, which is used to tackle multiple problems including VRP and scheduling problems. Wu et al. (2021b) applied Deep RL to learn improvement heuristics that iteratively employ local search-based operators for routing-based COPs including TSP and VRP with route capacity. Karimi-Mamaghan et al. (2023) integrated Q-learning into the iterated greedy metaheuristic as an efficient operator selection mechanism for the permutation flowshop scheduling problem.

Several recent studies focused on the integration of ML techniques with the classic LNS. As the first paper on this topic, Hottung and Tierney (2020) proposed two generalised random-based destroy operators and a single repair operator with automated learning based on a deep neural network with an attention mechanism. Their work was the first to consider the application of RL to LNS for solving VRP instances, and achieved solutions of better quality than classic optimisation approaches. Nevertheless, their proposed learning mechanism only focuses on repairing incomplete solutions during the repair phase. In another work, Falkner et al. (2022) integrated a pre-trained neural construction heuristic as the repair operator in the LNS framework to solve the VRP with time windows. The destroy procedures remain handcrafted and are classified into two groups without any learning involved. Oberweger et al. (2022) enhanced the LNS framework with an ML-guided destroy operator to solve a staff rerostering problem. For the reconstruction phase, the authors developed a mixed-integer linear program as a repair method. Lagos and Pereira (2024) proposed a multi-armed bandit algorithm for learning the online parameters that determine the selection of the next operator in the ALNS framework. Syed et al. (2019) proposed a neural network in an LNS setting to solve a vehicle ride-hailing problem. However, it uses Supervised Learning (SL), which requires a substantial training dataset and, furthermore, can only perform as well as the algorithm used for training data generation. Bongiovanni et al. (2022) also proposed a hybrid data-driven optimisation framework that contains an SL-guided operator classifier to predict the destroy–repair pair to improve an electric autonomous dial-a-ride problem. Although their framework can largely outperform the classic LNS and ALNS, the embedded SL technique requires training offline on a large labelled dataset to learn to select destroy–repair operator pairs, rendering it impractical when scalability of the method is a concern. Moreover, there is another set of works (Nair et al., 2020; Song et al., 2020; Wu et al., 2021a) combining learning with LNS. However, this line of research focuses on learning the neighbourhood selection policies for generalised (mixed-) integer programming instead of metaheuristic hybridisation or solving a specific COP heuristically.

Another orthogonal yet important aspect when applying ML to combinatorial optimisation is the learning representation that is used, i.e., the way the inputs are provided to and processed by the model. A classic learning architecture is the Multi-Layer Perceptron (MLP) neural network, which consists of layers of linear multiplications with learnable weight matrices, followed by applications of a non-linear activation function. While the MLP is demonstrably able to represent any continuous real function with sufficient parameters (Cybenko, 1989), it may not necessarily exploit problem structure effectively. Akin to the convolutional neural networks successful in the computer vision domain, GNNs (Scarselli et al., 2009; Sperduti and Starita, 1997) are learning architectures explicitly designed to operate on graphs and encode suitable inductive biases. In recent years, the graph ML community has explored the development of methods that provide different trade-offs in terms of expressibility and scalability (e.g., Dai et al. (2016), Hamilton et al. (2017), Veličković et al. (2018)). Advantages of GNNs that are worth mentioning in this context are their end-to-end learning of relevant features without requiring manual design, their permutation invariance, as well as their ability to operate on problem instances of different sizes (unlike the MLP).

Let us briefly refer to other works that, like the present paper, apply RL and GNNs to combinatorial optimisation. Beyond the already-discussed pioneering work of Khalil et al. (2017), similar methodologies have found applications in accelerating the column generation algorithm for solving linear programs (Chi et al., 2022), finding maximum independent sets in graphs (Ahn et al., 2020), and learning to construct graphs representing infrastructure networks (Darvariu et al., 2021) or molecules (You et al., 2018).

2.4. Comparison with concurrent works

Two very recent works by Kallestad et al. (2023) and Reijnen et al. (2024) both applied Deep RL to improve ALNS operator selection, using approaches that are similar to each other. In the remainder of this paper, we collectively refer to these approaches as Reinforcement Learning Baseline (RLB). Let us review the key differences with respect to them in this subsection.

Firstly, Kallestad et al. (2023) and Reijnen et al. (2024) both considered a state space that only uses high-level features related to the search status (e.g., the search step, the temperature, the performance gap from the best-found solution), while neglecting information linked to the current solution itself (e.g., the structure of the tours that make up the solution in a routing problem). In contrast, our method uses a substantially more expressive representation that focuses on the structure of the current solution itself to decide the choice of an operator.

Furthermore, we formulate the decision-making problem and train the operator selection policy independently of the ALNS process. This allows us to isolate the problem of selecting an optimal sequence of operators without the noise introduced by the search process (i.e., the occasional acceptance of a worse solution) and to benchmark different methods accurately. It also implies that our approach is not limited to the specific variant of ALNS that is considered, and can be applied transparently with other modifications and enhancements to the higher-level metaheuristic. Our approach also features a GNN as a learning representation, as opposed to the other works that opt for standard MLP architectures. The GNN enables our method to use granular information about the current solution while being applicable to instances of different sizes. This also allows the learned model to successfully generalise to unseen problem sizes, including larger instances.

Finally, an important aspect that differentiates our work is the scope and size of the considered operator portfolio, which contains 28 destroy and 7 repair operators. In comparison, Reijnen et al. (2024) used 4 destroy and 3 repair operators and Kallestad et al. (2023) opted for several destroy operators that are identical apart from the difference in destroy scales. Beyond the substantially larger portfolio size itself, a contribution of our work lies in the systematic review and integration of the destroy and repair heuristics that are found in the routing literature. As such, the experimental results we present are representative of the current state-of-the-art in this literature, and reflect the possible choices that a practitioner might face when deciding the operator portfolio for a given problem in the routing space.

3. Methodology

Our learning-based approach to improve the operator selection in ALNS consists, at a high level, of the following two steps. Firstly, we aim to isolate operator choice from the considerations of the SA process in ALNS, which introduces additional noise for navigating the solution space that may obscure the operators' contributions. To achieve this, we formulate operator selection for the COP as a standalone Markov Decision Process (MDP), in which an agent is given a limited *budget* of operators, and must learn to select those that lead to the best solutions. Secondly, the learned model is integrated into the ALNS loop and used to select operators in the SA process.

Each of the following subsections is dedicated to: reviewing the classic ALNS algorithm (Section 3.1); the proposed formulation of the task as an MDP (Section 3.2); the proposed RL+GNN method that enables learning operator selection policies for the MDP (Section 3.3); and, finally, the integration of RL models within the ALNS loop (Section 3.4).

3.1. Classic ALNS algorithm

In the classic ALNS framework proposed by Ropke and Pisinger (2006), the process begins by destructing and subsequently reconstructing an initial solution through the iterative application of an operator pair comprising a destroy operator $o_i^- \in \mathcal{D}$ and a repair operator $o_i^+ \in \mathcal{R}$, which together yield the new incumbent solution. The destroy scale d characterises the number of d nodes to be removed and re-inserted during an iteration, determining the extent of modifications. It can be randomly drawn or set as a hyperparameter.

In ALNS, the process is divided into *segments*, each of which contains a number of destroy–repair iterations. The number of iterations is referred to as *segment length*. A segment begins by assigning an initial score $\psi_i = 0$ to each operator o_i. The score is incremented by a value δ every time a new incumbent solution is generated using an operator pair that includes the specific operator o_i. The increase in score varies depending on the quality of the newly-found incumbent: the score is increased by δ_1 if the incumbent is a new global best solution, by δ_2 for a newly-found incumbent better than the previous solution but not the global best, and by δ_3 for an accepted yet worse newly-found incumbent, where $\delta_1 > \delta_2 > \delta_3$. As each segment K concludes, the cumulative score for each operator o_i and the number of times N_i it was selected within the segment are utilised to compute a corresponding weight $w_{i,K}$ for this segment, which serves as an estimate of the operator's capability to discover promising solutions. As shown in Eq. (1), weights for the next segment $K+1$ are determined using a weighted average of the historical weight and its average performance in segment K:

$$w^{\pm}_{i,K+1} = \begin{cases} (1-\alpha_{\text{RW}}) \cdot w^{\pm}_{i,K} + \alpha_{\text{RW}} \cdot \frac{\psi_i}{N_i} & \text{if } \psi_i > 0, \\ w^{\pm}_{i,K} & \text{if } \psi_i = 0, \end{cases} \tag{1}$$

where the reaction factor α_{RW} weights the influence of operator o_i's performance in the current segment K versus its historical performance. Within each iteration of the segment, the RW selection algorithm is employed to select a pair of destroy and repair operators based on their associated probabilities, computed by $w^-_{i,K} / \sum_{j\in\mathcal{D}} w^-_{j,K}$ and $w^+_{i,K} / \sum_{j\in\mathcal{R}} w^+_{j,K}$ respectively. Initially, all operators are assigned an equal weight and hence share an identical selection probability to ensure that each operator is given an equal opportunity to contribute to the search at the beginning. Throughout the search process, the operators accumulate different scores and their selection probabilities evolve to reflect their respective performance. Consequently, the selection algorithm is adjustable to the changing landscape of the search process by assigning higher probabilities to operators that have exhibited more robust capability, hence improving the general efficiency of the algorithm.

Upon the formation of a new solution, the ALNS algorithm features an acceptance mechanism (typically, the SA metaheuristic) to determine whether the newly-formed solution should be accepted as the new incumbent for the subsequent iteration. The mechanism aims to diversify the search by allowing the probabilistic acceptance of worse solutions with the potential for further improvement, and therefore reduces the chance of the search process becoming trapped in local optima and terminating prematurely. The SA mechanism embodies a probabilistic methodology that resembles the steel annealing process wherein an increase or decrease of temperature corresponds to a higher or lower acceptance probability of inferior solutions. Therefore, this facilitates the exploration of the solution space towards the globally optimal solution. Mathematically, the new solution s' is accepted with probability $\min(1, \exp\{{}^{[F(s')-F(s)]}/_{\tau}\})$ where F is the objective function in a minimisation problem, s is the current solution, and s' is the newly-found solution. The temperature parameter τ is initially assigned a positive value, which progressively decreases at a predetermined rate converging to 0 by multiplying its preceding value with a cooling coefficient $c_\tau < 1$. This results in an increasingly stringent mechanism that is more resistant to accepting worse solutions towards the end.

The search process continues until specific stopping criteria are met, including a predefined number of iterations, maximum computation time constraints, or the attainment of a satisfactory solution quality to ensure that the algorithm terminates appropriately while providing a solution meeting the desired requirements.

3.2. Operator selection as a Markov decision process

3.2.1. Markov decision process fundamentals

As described previously, we aim to use RL in order to discover sequences of operators that yield the best gains in the objective function. To do so, we must first cast this problem as a MDP, the mathematical model of decision-making with discrete time steps that underpins RL.

Let us first define some terminology. An MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, R)$ consisting of the state space, action space, transition function and reward function that fully defines the interaction of the decision-maker (also called *agent*) with an uncertain environment. A state S_t is a possible configuration of the environment at time t (e.g., a solution as defined by the given COP). In each state S_t of the possible state space $\mathcal{S}$, the agent selects an action A_t from the set of valid actions $\mathcal{A}(S_t)$ and, consequently, receives a reward signal R_t according to a reward function $R(S_t, A_t)$. Afterwards, the agent transitions to a new state S_{t+1} that depends on the conditional probability $P(s' = S_{t+1} | s = S_t, a = A_t)$, which governs the environment dynamics and is dependent only on the preceding state and action, for all $s', s \in \mathcal{S}$, $a \in \mathcal{A}(s)$.

The sequence of agent-environment interactions defines a *trajectory* of the form $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots, S_T$ that continues until a terminal state S_T is reached. We also define a *policy* $\pi(a|s)$ that specifies the probability of choosing action a in state s, which completely specifies the behaviour of the agent. The *state–action value function* $Q(s, a)$ is the expected sum of rewards that the agent will receive by picking action a in state s, then continuing to follow π. The goal of RL is to find the optimal policy π^* which maximises the expected cumulative reward across all possible trajectories in the MDP.

3.2.2. Problem formulation

At a high level, our MDP formulation provides the agent with a starting state S_0 corresponding to an initial solution, and a fixed *training operator budget* B_{MDP} of pre-defined destroy and repair operators that can be applied to modify the solution. The agent alternates between selecting a destroy and a repair operator until the budget is exhausted. At the final step, the agent receives a reward corresponding to the improvement in the objective function compared to the initial solution.

Our framework is designed to be applied to COPs that can be represented as graphs. Therefore, we assume that we are given an undirected graph $G = (V, E)$ defined by the given COP, a feature matrix $\mathbf{X}^{\text{node}}$

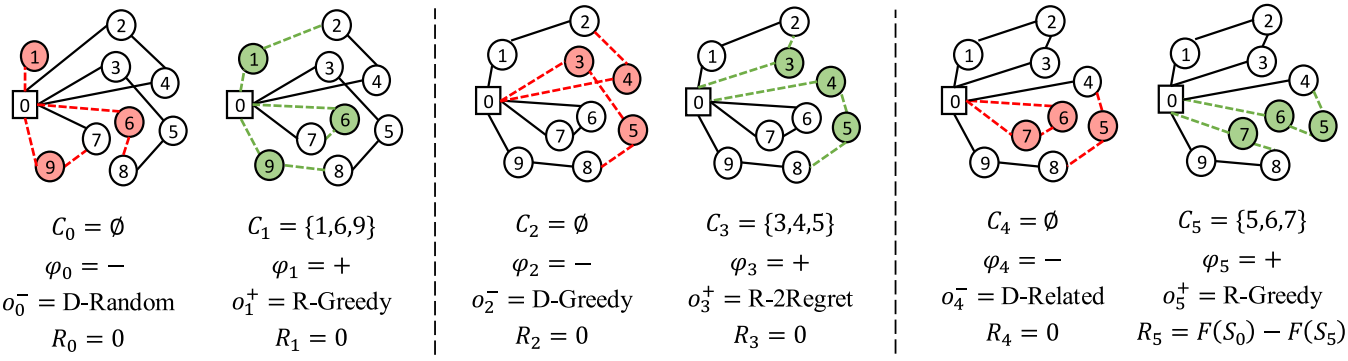


Fig. 1. Illustration of an MDP episode with budget $B_{\text{MDP}} = 6$ and destroy scale $d = 3$. The agent selects 3 action pairs before exhausting its budget and reaching the terminal state.

in which each row contains information about the nodes (e.g., node coordinates, node demand), and a feature matrix $\mathbf{X}^{\text{edge}}$ in which each row contains information about the edges (e.g., pairwise distances). These elements remain static throughout the course of execution and are not affected by the application of destroy and repair operators.

In contrast, the dynamic elements of a state are indexed by the time step t and change as operators are applied. J_t is the current, possibly partial solution to the COP. C_t contains intermediate information that is tracked to ensure solutions are valid. φ_t indicates the *phase*: whether the set of destroy or repair operators is eligible to be applied at time t. Finally, b_t indicates the operator budget available to the agent at time t.

Tasks are *episodic*, beginning with a starting state S_0 sampled from a distribution of initial solutions, and ending at terminal states when the operator budget is exhausted (i.e., $b_t = 0$). The environment subsequently resets, and the process begins with a different S_0. Let us now formulate each of the MDP elements using the classic VRP, without going into details of each variant.

- *States* $\mathcal{S}$: each state S_t is a tuple $(G, \mathbf{X}^{\text{node}}, \mathbf{X}^{\text{edge}}, J_t, C_t, \varphi_t, b_t)$. As mentioned above, G, $\mathbf{X}^{\text{node}}$, and $\mathbf{X}^{\text{edge}}$ remain constant. J_t is the set of node visiting sequences (referred to as *tours*) that start and end at the depot, forming the solution at time t. Notice that J_t also represents partial solutions formed after destroy and before repair phase. The intermediate information $C_t = V \setminus J_t$ holds all the d nodes temporarily removed from the solution that will be reinserted at the next step, where the destroy scale d is a non-negative integer not exceeding $|V|$.
- *Actions* $\mathcal{A}$ involve the selection of an operator $o_t^{+/-}$, with those available defined as a subset of $\mathcal{D}$ if $\varphi_t = -$ during the destroy phase, and $\mathcal{R}$ if $\varphi_t = +$ during the repair phase. Operator choices that are infeasible are forbidden.
- *Transitions* P apply the selected operator $o_t^{+/-}$ to the current solution. Applying a destroy operator removes d nodes from the tours J_t and places them in the removal list C_t. Using a repair operator reinserts the nodes from C_t into J_t, leaving C_t empty and the solution J_t complete. The transition model also decreases the operator budget by 1 with each time step. Transitions are stochastic due to the inherent randomness of the operators.
- *Rewards* R are provided once the operator budget is exhausted (i.e., the modifications dictated by the final repair operator are applied) and the improvement in solution quality can be assessed via an objective function F. Concretely, $R(S_t, A_t) = F(S_0) - F(S_t)$ (minimisation problem) if $b_t = 0$, and 0 otherwise, where S_0 is the starting state of each episode.

We illustrate the key features of our problem formulation in Fig. 1, which shows a visualisation of an episode in the MDP. We assume a training operator budget $B_{\text{MDP}} = 6$ and a destroy scale $d = 3$. Depending on the specific phase of the operator ($\varphi_t = -$ or $\varphi_t = +$), the agent performs either the removal or insertion of 3 nodes into the solution. The action spaces contain 3 destroy operators $\mathcal{D} = \{\text{Random, Greedy, Related}\}$ and 2 repairs $\mathcal{R} = \{\text{Greedy, 2Regret}\}$. The agent begins at state S_0 with customer pool $C_0 = \emptyset$ and set of tours $J_0 = \{[1], [2, 4], [3, 5, 8, 6], [7, 9]\}$. The agent selects the destroy operator o_0^- = Random in the destroy phase with $\varphi_0 = -$. At this state, nodes 1, 6 and 9 together with their linking arcs are removed (marked as red) and placed into the temporary customer pool. Subsequently, the agent selects the repair operator o_1^+ = Greedy at the repair phase $\varphi_1 = +$ by reinserting the removed nodes back into the tours (marked in green) using the instruction specified by o_1^+ and fixing the linking arcs to form a complete tour, thus transitioning to a new state S_2. By completing one destroy–repair iteration, the number of tours goes from 4 to 3. The episode continues with the selection of operators until the budget is exhausted and the terminal state S_5 with set of tours $J_5 = \{[1, 2, 3], [4, 5, 6], [7, 8, 9]\}$ is reached. Finally, it receives a reward $R_5 = F(S_0) - F(S_5)$ proportional to the improvement in solution quality, where all the non-terminal rewards $R_0 = \cdots = R_4 = 0$.

3.3. Learning an operator selection policy with DQN and GNNs

Given the MDP problem formulation above, in principle any RL algorithm is applicable for learning the policy π for operator selection. In this work, we opt for the Deep Q-Network (DQN) algorithm (Mnih et al., 2015), a value function based method, as is also done by other works combining RL and GNN representations (Khalil et al., 2017; Darvariu et al., 2021; Chi et al., 2022). An advantage of this class of methods over policy gradient approaches (e.g., as used in concurrent works (Reijnen et al., 2024; Kallestad et al., 2023)) is the superior sample complexity, which means that substantially fewer training steps are required to reach a well-performing policy (see, e.g., Chua et al. (2018)). A possible disadvantage is that the DQN learns a greedy policy, which means that it will always select the action with the highest expected reward. However, our preliminary work (Johnn et al., 2023) has demonstrated that greedy policies perform well for operator selection in this setting, as the inherent stochasticity of the operators is sufficient to achieve good exploration of the search space.

We refer to our method comprising the MDP formalisation and the RL+GNN learning mechanism as *Graph Reinforcement Learning for Operator Selection* (GRLOS). In the following subsections, we provide some further technical details about its components. Finally, we also give a comparison between Q-learning based methods and the classic RW update. This leads to a simple yet effective method we call Learned RW (LRW).

3.3.1. Q-learning

Q-learning (Watkins and Dayan, 1992) is a model-free RL approach used for solving MDPs that relies on estimating the previously defined state–action value function $Q(s, a)$, also known as Q-values. A policy π can be derived by selecting actions greedily with respect to their associated Q-values at a particular state. The agent's interactions with

the environment generates tuples of the form (s, a, r, s'), and the Q-value estimates are iteratively updated according to the following rule:

$$Q(s,a) \leftarrow (1-\alpha_{\text{RL}}) \cdot Q(s,a) + \alpha_{\text{RL}} \cdot \left(r + \gamma \cdot \max_{a' \in \mathcal{A}(s')} Q(s',a') \right) \tag{2}$$

where $Q(s,a)$ is the Q-value for a particular state–action pair (s,a), r denotes the immediate reward after taking action a in state s, and max $Q(s',a')$ represents the maximum Q-value amongst all the available actions in the subsequent state s'. Furthermore, α_{RL} denotes the learning rate that balances the information from the current and future states, and γ is a discount factor that trades immediate versus long-term rewards. The repeated application of this rule is known to lead to the optimal action-value function and optimal policy at convergence (Watkins and Dayan, 1992).

3.3.2. Function approximation and graph neural networks

Many realistic problems exhibit very large state spaces, which makes the application of standard tabular Q-learning intractable. This is often the case for COPs, in which it is impossible to enumerate all possible solutions and store them in memory beyond problems of a trivial size. For such problems, deep neural networks are commonly employed for *function approximation* in order to estimate the $Q(s,a)$ function. Let us denote this estimate by $\hat{Q}(s,a;\theta)$, where θ is the set of learnable parameters of the neural network. Function approximation is a powerful technique to generalise across states of the environment that, while not identical, share common characteristics and will lead to similar expected rewards.

The DQN algorithm (Mnih et al., 2015), which uses function approximation together with replay buffers and target networks, has been successfully applied for a variety of decision-making problems with large state spaces. The use of replay buffers means that (s, a, r, s') tuples encountered by the agent are stored in a memory bank, and updates to the parameters θ are performed periodically by sampling from this buffer, instead of online. The target network mechanism stores a separate copy of the parameters θ', which are updated periodically by copying the main network parameters θ, and kept fixed at every intermediate step. This increases the stability of the algorithm by mitigating the "moving target" problem – given that the learning target depends on the network weights, which are updated with every learning step, the optimisation objective is constantly changing. Keeping the weights fixed for a number of step results in a series of well-defined optimisation problems instead (Mnih et al., 2015). The network weights are updated by stochastic gradient descent to minimise the mean squared error between the predicted $\hat{Q}(s,a;\theta)$ and the learning target $r + \gamma \max_{a'} \hat{Q}(s',a';\theta')$.

Given our focus on problems that can be captured as a graph, we require a suitable learning representation. As we have argued in Section 2, GNNs are explicitly designed to operate on graph-structured data. At a high level, given features $\mathbf{x}_i^{\text{node}}, \mathbf{x}_i^{\text{edge}}$ for each node and edge in the graph, GNNs compute an embedding vector $\boldsymbol{\mu}_i$ that encode individual node features and its structural relationship to other nodes within the graph. This is achieved by applying several message-passing layers that aggregate the features of neighbouring nodes and apply a non-linear activation function. From the node-wise embeddings, representations of the entire graph can be obtained, for example, by a sum aggregation: $\sum_i \boldsymbol{\mu}_i$. Concretely, in this work we opt for the Graph Attention Network (GAT) (Veličković et al., 2018), which performs a flexibly-weighted aggregation via the attention mechanism.

One notable advantage of GNNs is that their parametrisation θ (i.e., the union of all neural network parameters in all the layers) is independent of the number of nodes in the graph. This implies that GNNs enable learning an approximation of the state–action value function on small instances and applying it directly on large instances, which is an appealing approach for COPs (Bengio et al., 2021).

3.3.3. Comparison of Q-learning and Roulette Wheel

By comparing the one-step Q-learning update rule in Eq. (2) to the classic RW update rule in Eq. (1), we notice that both use a weighted factor to balance two terms representing the historical and current estimates of algorithm performance. In the classic RW framework, the weight of each operator can be interpreted as the operator's ability to improve the solution quality for the current segment containing multiple destroy–repair iterations. These are periodically updated using a weighted balance of the operator's historical and current performance at the end of each segment. As search progresses across segments, each operator's selection probability is directly proportional to its weight, leading to the more frequent selection of operators with a higher weight. As a result, the ALNS adaptive layer reflects each operator's capability at a macroscopic level by sampling operators according to the fixed probabilities within a segment, as mentioned in Kallestad et al. (2023).

We observe that the Q-learning estimate is conditioned on the state. This implies that the Q-learning algorithm can consider granular state information while, in contrast, the RW approach simply averages the gains obtained by the operators within the segment. This can be interpreted as a rough approximation of the Q-learning update, tracking the average reward from all the possible trajectories where an operator is included regardless of the state-based performance. Consequently, the Q-learning update formula can more accurately guide the future action selection process by tracking each operator's performance at the individual state level. This nevertheless implies that Q-learning requires higher sample complexity. However, we found this was not a significant concern in practice, as a relatively small number of training steps suffices to achieve a good policy.

Beyond this theoretical insight, we would like to quantify the extent of the improvement of RL over the simplified RW update. However, the RW mechanism is dependant on the progress of ALNS and is not well-defined for the standalone operator selection MDP. Furthermore, while the CRW mechanism is appropriate for optimising a single instance fully on-line, it is not suitable for an off-line setting. The weights that are computed on the off-line training instances would be misaligned when evaluated for unseen instances given the weights were calculated relative to a different incumbent solution and global optimum, which will vary depending on the stage of the search as well as the specific problem instance.

To make the RW mechanism applicable in the MDP setting and compatible with an off-line training stage, we make the following two adaptations to obtain a method we call *Learned Roulette Wheel* (LRW). Firstly, in Eq. (1), we replace the manually-defined operator scores ψ computed from the discretised δ with the continuous objective value F for the reward function to more accurately reflect the solution quality. Secondly, we also adjust the reward feedback frequency from every operator pair in RW to every episode in the MDP.

Preliminary experimental results suggested that the CRW outperforms the LRW by less than 2% when applied in ALNS without any prior training, reflecting the coarser weight updates of LRW. However, the advantage of LRW is that its weights can be learned off-line in a prior training stage, and therefore the method can start from an informative set of weights rather than from scratch when faced with a new problem instance. As we shall demonstrate in our experimental evaluation, this leads to the LRW significantly outperforming the CRW across the board.

3.4. Integration of RL with ALNS

As mentioned at the start of this Section, during the first stage of our learning-based approach, we isolate the operator selection process from the ALNS framework and convert it into an MDP for training the agent. The second stage involves integrating the fully-trained model into the ALNS algorithm to search for good-quality solutions.

The following methodological conundrum is worth noting regarding the integration of RL and ALNS, which applies not just to our method,

but more widely to the use of any ML approach with this metaheuristic. Namely, it is the case that, after a certain point in the ALNS process, the model will be asked to select operators for solutions that are dissimilar to those seen during training. This is by definition since, if we knew how to select operators optimally at every step, the optimal solution reached at the end of the process would be part of the training set and we would not need to use ALNS to begin with for exploring the solution space. This phenomenon, known as *distribution shift*, occurs when the distributions used to generate samples for training and testing differ significantly. It is known to greatly impact the performance of ML models (Quiñonero-Candela et al., 2008).

We thus distinguish between the *training operator budget* B_{MDP} and the *ALNS operator budget* B_{ALNS}, with $B_{\text{ALNS}} > B_{\text{MDP}}$. In this context, we refer to the states that can be encountered when applying operators within the defined training operator budget B_{MDP} as *in-distribution*. One may continue to apply operators beyond this point until B_{ALNS} is exhausted, but the model may suffer in performance since it will encounter states dissimilar to those it was trained on. We refer to this setting as *out-of-distribution*.

We therefore propose the following partial mitigation that applies in an out-of-distribution setting, where the RL algorithm's predictions of future rewards become inaccurate due to distribution shift. We compute the frequencies with which the operators were applied on the training set when the model is in-distribution; then, beyond this point, operators are sampled proportionally to these frequencies. We denote this approach as *GRLOS-M*, as the operator selection probabilities are made to *match* those observed during the training operator budget. Similarly, for the RLB baseline, we denote this variant as *RLB-M*. As we will demonstrate in our experimental evaluation, this mitigation greatly improves the performance of RL models when used in ALNS.

The LRW model, which only tracks simple statistics, is substantially simpler and not subject to this caveat. Accordingly, we also consider a *Hybrid* policy in our evaluation, which combines the strengths of GRLOS and LRW. This method uses the GRLOS policy within the training operator budget and switches to the LRW policy once the RL model is out-of-distribution.

4. Experimental setup

4.1. VRP variants, datasets, and initial solutions

Tackling NP-hard COPs by classic heuristic techniques most often requires the repeated handcrafting of rules, even when similar problem structure is manifested. Our proposed learning-based method exploits this recurring effect by automating the process, making the same approach transparently applicable to different problem variants and datasets. To demonstrate this wide applicability, we evaluate our method on 5 different routing problems:

- *Travelling Salesperson Problem (TSP)*: Solutions consist of a single vehicle serving all customer nodes within a single grand tour with no vehicle capacity restrictions. The objective is to minimise the total traversal cost.
- *Capacitated Vehicle Routing Problem (CVRP)*: The goal is forming multiple tours from a single depot served by a fleet of homogeneous vehicles of fixed capacity, each visiting an individual group of customer nodes. The capacity restriction applies to the total carrying load of vehicles.
- *Heterogeneous Vehicle Routing Problem (HVRP)*: The solution is formed of multiple tours from a single depot served by a fleet of heterogeneous vehicles, where each vehicle type is associated with a given fixed capacity and a finite number of available vehicles. The objectives are to minimise the total traversal and vehicle hiring costs.
- *Vehicle Routing Problem with Time Windows (VRPTW)*: The goal is to establish multiple tours for a homogeneous fleet of vehicles from a single depot. The customer node arrival time must be within the a-priori customer time window to avoid an additional penalty. The objectives are to minimise the total traversal, vehicle hiring, and out-of-time-window penalty costs.
- *Location-Routing Problem (LRP)*: Given a set of potential depot locations, jointly consider the opening of a selected subset of depots, the allocation of each customer node to an open depot, and forming tours using a finite set of homogeneous vehicles to serve the customers from the selected depot. The objectives are to minimise the total traversal, vehicle hiring, and depot opening costs.

One reason for choosing these problems is that they are some of the most studied COPs, enabling us to leverage a large body of literature on heuristic operator design. Furthermore, solutions to these problems can naturally be represented as graphs: the tours making up the solution are a collection of cyclic paths on the fully-connected graph. More broadly speaking, our methodology applies to any COP that can be naturally modelled as a graph, a mathematical structure that is pervasive in combinatorial optimisation, and thus aligned with the objective of designing ML approaches for a broad class of related problems (Bengio et al., 2021).

We use the R, C and RC instances (random, clustered, and mixed random-clustered nodes) of the classic Solomon dataset (Solomon, 1987), each containing a depot and up to 100 customers. Up to 3 depots are provided for the LRP variant with 35 customers per depot. We assign the vehicle capacity to be 200, and adjust it proportionally if scaling down the instance to fewer customers. Furthermore, we also consider the larger-scale Gehring and Homberger (GH) instances (Gehring and Homberger, 1999) with up to 1000 nodes. The GH dataset follows a similar structure to the Solomon dataset with the same R, C, RC categories but with different node locations, demands and instance sizes.

In terms of initial solutions that are provided to the ALNS process, we primarily consider randomly generated initial sets of tours whose length is proportional to the defined vehicle capacity (therefore the instance size). An exception is the TSP problem, which does not have a notion of vehicle capacity. Instead, a randomly shuffled sequence consisting of all nodes is generated as the initial solution. The application of randomised initial solutions for all the considered routing problems was made so that the performance gaps between the different operator selector strategies are emphasised in the evaluation. Nevertheless, the methodology is compatible with any other means of initial solution generation.

As a comparison, we also generate initial sets of tours using a self-implemented Clarke-Wright saving algorithm with noise, denoted "noisy Clarke-Wright", to construct relatively good-quality initial solutions. The noisy Clarke-Wright algorithm adheres to the procedure given in the work of Clarke and Wright (1964), with the actual travelling distance between each pair of nodes multiplied by a noise factor randomly selected from $[0.8, 1.2]$. The inclusion of noise ensures that the initial solutions are not identical, which is critical for a meaningful evaluation of learning-based approaches, while still starting the search process within a good neighbourhood. We also note that this noise factor is only multiplied with the distance matrix used to formulate the initial solution, and not to any solution formed during the training or evaluation process.

4.2. Considered operators

All operators considered in this study already exist in the literature. We implement their mechanisms with minor adaptations to fit the specific COP environment, without fine-tuning the operator design to boost performance. In this way, the list of operators can yield varying performances in the given model, some of which may even bring down

Table 1
Destroy operators used for the different routing problems, as indicated by ✓.

Features	Random	Greedy	Related	TSP	CVRP	HVRP	VRPTW	LRP
Distance-based	Random Node, Random Route	Worse Distance, Route Deviation, Greedy Route Longest, Greedy Route Shortest	Proximity Distance, Shaw, Cluster, Pair, Zone, Node Neighbourhood, Route Neighbourhood	✓	✓	✓	✓	✓
Historical-based	Forbidden Random	Historical Knowledge	Historical Pair	✓	✓	✓	✓	✓
Demand-based		Largest Demand Deviation, Smallest Demand, Largest Demand	Proximity Demand		✓	✓	✓	✓
Time Window-based		Worst Time	Proximity Time, Proximity Service				✓	
Facility-based	Close-an-open-Facility, Open-a-closed-Facility	Worst Facility Node	Facility Swap					✓

the solution quality in general. However, an effective agent should be capable of avoiding the operators that harm the solution improvement process without human intervention.

The sets of destroy operators applied to each problem variant, summarised in Table 1, vary due to differences in individual neighbourhood structures. We select 27 popular destroy operators from the ALNS literature spanning the representative categories and include an additional no-action operator that leaves the solution unchanged. The repair operators remain the same for all problem variants. Here we use random repair, greedy repair, greedy perturbation repair, deep greedy repair, and k-regret repair ($k = 2, 3, 4$).

4.3. Training and evaluation methodology

For each instance, we generate 3 distinct sets of initial solutions $\mathcal{J}^{\text{train}}, \mathcal{J}^{\text{validate}}, \mathcal{J}^{\text{test}}$ that are used for model training, parameter tuning and model selection, and final evaluation respectively. $\mathcal{J}^{\text{train}}$ and $\mathcal{J}^{\text{test}}$ contain 128 sets of tours with each of $[20, 30, 40, 50, 60, 70, 80, 90, 100]$ customers, yielding 1152 sets of tours in total. The validation set $\mathcal{J}^{\text{validate}}$ is smaller in comparison and contains 64 sets of tours with each of $[20, 30, 40, 50, 100]$ customers, yielding 320 sets of tours in total. This enables meaningful model selection while being sufficiently small to assess performance quickly during the training process.

The proposed GRLOS and LRW methods, as well as the RLB baseline, undergo prior training using initial solutions from $\mathcal{J}^{\text{train}}$ for $5 \cdot 10^4$ steps. Periodically (i.e., every 25 steps), the generalisation capability of the model is evaluated on the hold-out set $\mathcal{J}^{\text{validate}}$, and the best-performing model found during training is stored and used for reporting the final results. Models are trained separately for each instance type (C/R/RC) and problem variant, but must generalise across instance sizes and different unseen initial solutions. For GRLOS and RLB, each training step consists of (1) applying a sequence of operators until the training operator budget B_{MDP} is exhausted; (2) adding the resulting (s, a, r, s') experience entries to the replay buffer; and (3) sampling a batch of experience entries and updating the neural network parameters θ by Stochastic Gradient Descent (SGD) using the Q-learning loss. LRW similarly executes (1) then, akin to (3), updates its per-operator set of weights in an online fashion computed via Eq. (1).

There are two "modes" that we consider in our evaluation. The first (*MDP Evaluation*) benchmarks the quality of operator selection techniques in the MDP, independently of the ALNS process and occurs over B_{MDP} steps, in which the learning-based models are in-distribution. The proposed GRLOS and LRW methods, the RLB baseline, and a uniform random sampling (RAN) of operators are considered in this setting. The second mode (*ALNS Evaluation*) considers the full ALNS task, which occurs over a larger number of B_{ALNS} steps and also features the SA acceptance mechanism. For this mode, we additionally consider the CRW agent (which does not require training and can only be used within ALNS), the GRLOS-M and RLB-M methods that contain the out-of-distribution mitigation, and the Hybrid method that combines GRLOS and LRW.

Training (where applicable) and evaluation are repeated across 10 random seeds for all agents, which are used to compute 95% confidence intervals. We note that all reported figures include confidence intervals, which may not always be visible due to their tightness.

4.4. Features for representing state in RL agents

Let us now discuss the features that are used by GRLOS and RLB to represent the state, as well as details of the other components used for training. We distinguish between three types of features: *node* features correspond to each node in the solution, *edge* features are attached to arcs, and *global* features characterise the current solution at a high level. Furthermore, features can also be *static*, meaning they do not change with modifications to the solution (as is the case, e.g., for the geographical coordinates of the depot in routing problems), or *dynamic* otherwise.

The features used by GRLOS are shown in Table 2. Since each VRP variant is characterised by different constraints, the feature representation differs slightly between the variants. Notably, we exclude the *tour_id* for TSP and include an additional set of time-related node features *tw_start*, *tw_end*, and *service_time* for VRPTW. The node features for all the potential facilities are also included inside the feature matrix for LRP. The remaining variants follow the same node features.

In contrast, the features used by the RLB baseline are shown in Table 3, and are in essence a union of the features used by Reijnen et al. (2024) and Kallestad et al. (2023), minus the few features that are undefined outside of the ALNS process (such as the temperature and cooling schedule). We note that all features used by this baseline are global and dynamic. Furthermore, given the dependencies of transitions and rewards not only on the current state but also past states implied by these features, we note that the Markov assumption underlying the MDP formalism is not satisfied. This may harm performance given the reliance of standard RL algorithms on this assumption.

For GRLOS, we use the node and edge features as input to the GNN, and use sum aggregation to compute an embedding for the entire graph. This is then concatenated with the global features to obtain the state representation. Concretely, as previously stated, we opt for the GAT (Veličković et al., 2018). We use 3 message-passing layers and node embedding dimension equal to 32. Both GRLOS and RLB use an MLP architecture consisting of hidden layers and ReLU activations, with the final layer containing one output unit for each valid destroy or repair operator. The MLP has 64 units in the first hidden layer, with the subsequent layers having half the size.

Table 2
Features used by GRLOS to represent state.

Feature	Type	Dynamicity	Description
x, y	Node	Static	Geographical coordinates for each customer node.
demand	Node	Static	Physical demand associated with each customer node.
tour_id	Node	Dynamic	The index of the tour this node belongs to, represented as an integer (excluded for TSP).
pos_id	Node	Dynamic	The sequential index of each node within each tour as an integer.
tw_start	Node	Static	The start time of the customer time window (VRPTW only).
tw_end	Node	Static	The end time of the customer time window (VRPTW only).
service_time	Node	Static	The service time at each customer node (VRPTW only).
adj_dis	Edge	Dynamic	The Euclidean distance from each node to its subsequent neighbour in the tour.
inst_size	Global	Static	The number of customer nodes in the current instance.

Table 3
Features used by the competing RLB baseline (Reijnen et al., 2024; Kallestad et al., 2023) to represent state.

Feature	Type	Dynamicity	Description
reduced_cost	Global	Dynamic	Cost reduction between previous and current solution.
cost_difference_best	Global	Dynamic	Difference between objective value of current and best solution.
cost_of_best	Global	Dynamic	Cost of best encountered solution.
is_current_best	Global	Dynamic	Whether the current solution is the best solution (0 or 1).
cost	Global	Dynamic	Cost of the current solution.
iteration	Global	Dynamic	Current iteration (i.e., the time step).
stagnation_count	Global	Dynamic	Number of iterations without improving the best found solution.
unseen	Global	Dynamic	1 if solution was not previously encountered, and 0 otherwise.
was_changed	Global	Dynamic	1 if solution was changed from the previous one, and 0 otherwise.
last_action_sign	Global	Dynamic	1 if the last action improved the solution, and 0 otherwise.
last_action	Global	Dynamic	One-hot encoding of the last taken action.

4.5. Operator selector and ALNS parameters

For all experiments, we use a destroy scale $d = |V|/10$ unless otherwise stated. Both RL methods use identical parameters: a learning rate of $\alpha_{\text{RL}} = 0.0005$ and SGD with the Adam optimiser and a batch size of 16. The exploration rate ϵ is linearly decayed from 1 to 0.1 in the first 10% of steps, then remains fixed. The replay buffer size is equal to 3% of the number of steps.

When applied in the ALNS framework, all methods use the SA acceptance criterion. Regarding ALNS parameters, we set the cooling coefficient $c_\tau = 0.998$, the initial temperature $\tau = 100$, and the non-changing iteration stop criterion to be 1000. We use a segment length of 200 and reaction factor $\alpha_{\text{RW}} = 0.99$ for all methods except CRW, whose parameters are tuned for fairness as described below. For CRW, we set the scores $\delta_1, \delta_2, \delta_3$ to be $2, 1, 0$ respectively. Unless otherwise stated, we execute ALNS for $B_{\text{ALNS}} = 100$ steps.

For fairness of comparison with methods that have undergone prior training, we allow a tuning of parameters for the CRW method. We consider values for the segment length in $\{20, 75, 200\}$ and reaction factor in $\{0.9, 0.95, 0.99\}$. The optimal parameter configuration out of the 9 possible ones is chosen separately according to performance on $\mathcal{J}^{\text{validate}}$ for each type of instance and problem variant. We do not carry out parameter tuning for the other methods beyond the reported values due to computational infrastructure constraints.

5. Computational study

5.1. MDP evaluation

We first evaluate the models in the *MDP Evaluation* mode, in which the methods are benchmarked for their ability to select the best sequence of $B_{\text{MDP}} = 20$ operators independently of the ALNS process. From Fig. 2, we observe that the learning-based GRLOS, LRW and RLB models are able to achieve good cumulative reward and possess generalisation to varying instance sizes and initial solutions. All learning-based methods outperform uniform random operator selection (RAN) statistically significantly. For all the 5 routing problems, GRLOS is able to accumulate a larger sum of rewards than LRW and RLB. The only exception occurs for the R instance and LRP, where LRW achieves better average results than GRLOS with instance sizes larger than 80. However, the confidence intervals for GRLOS include the LRW performance, and the best reward accumulated by GRLOS is still higher than that of LRW.

5.2. ALNS evaluation

Furthermore, we evaluate the performance of the models within ALNS, in which the number of operator selections to be performed ($B_{\text{ALNS}} = 100$) exceeds the training operator budget $B_{\text{MDP}} = 20$. We additionally include the GRLOS-M, Hybrid, and CRW methods as previously discussed. For visual compactness, the comparison with the RLB baseline is deferred to Section 5.5.

From Fig. 3, we observe an overall performance hierarchy among the models that follows the pattern of GRLOS-M, LRW, and Hybrid models exhibiting better overall performance compared to the vanilla GRLOS, followed by CRW, and finally the RAN baseline. In TSP and LRP, we notice a sharp decline in the GRLOS model performance level compared to the other methods above 70 customers. However, the other GRLOS-based models (GRLOS-M and Hybrid) show no similar trend. The reason behind the sudden performance drop for standard GRLOS in some cases (most notably, LRP above 70 customers) is due to the previously mentioned distribution shift limitation of ML approaches. These results, however, show that the proposed GRLOS-M mitigation is effective, as it leads to improvements in every setting tested. A jump in objective value from 70 to 80 customers is observed in the LRP variant for all methods, the reason for which is the inclusion of an additional depot to serve the total customer demands, which causes the objective function values of the instances to increase sharply.

The overall improvement in solution quality over the RAN baseline is summarised in Table 4, averaged across all sizes of a problem variant and instance type respectively. On average, the Hybrid method results in the best overall improvement over uniform RAN method, with the exception of the simpler TSP problem, on which LRW performs slightly better.

5.3. ALNS evaluation with Clarke-Wright initial solutions

For this experiment, also carried out in the ALNS Evaluation mode, we aim to assess the agents' performance based on initial solutions of different quality. We focus exclusively on the RC instance in this section

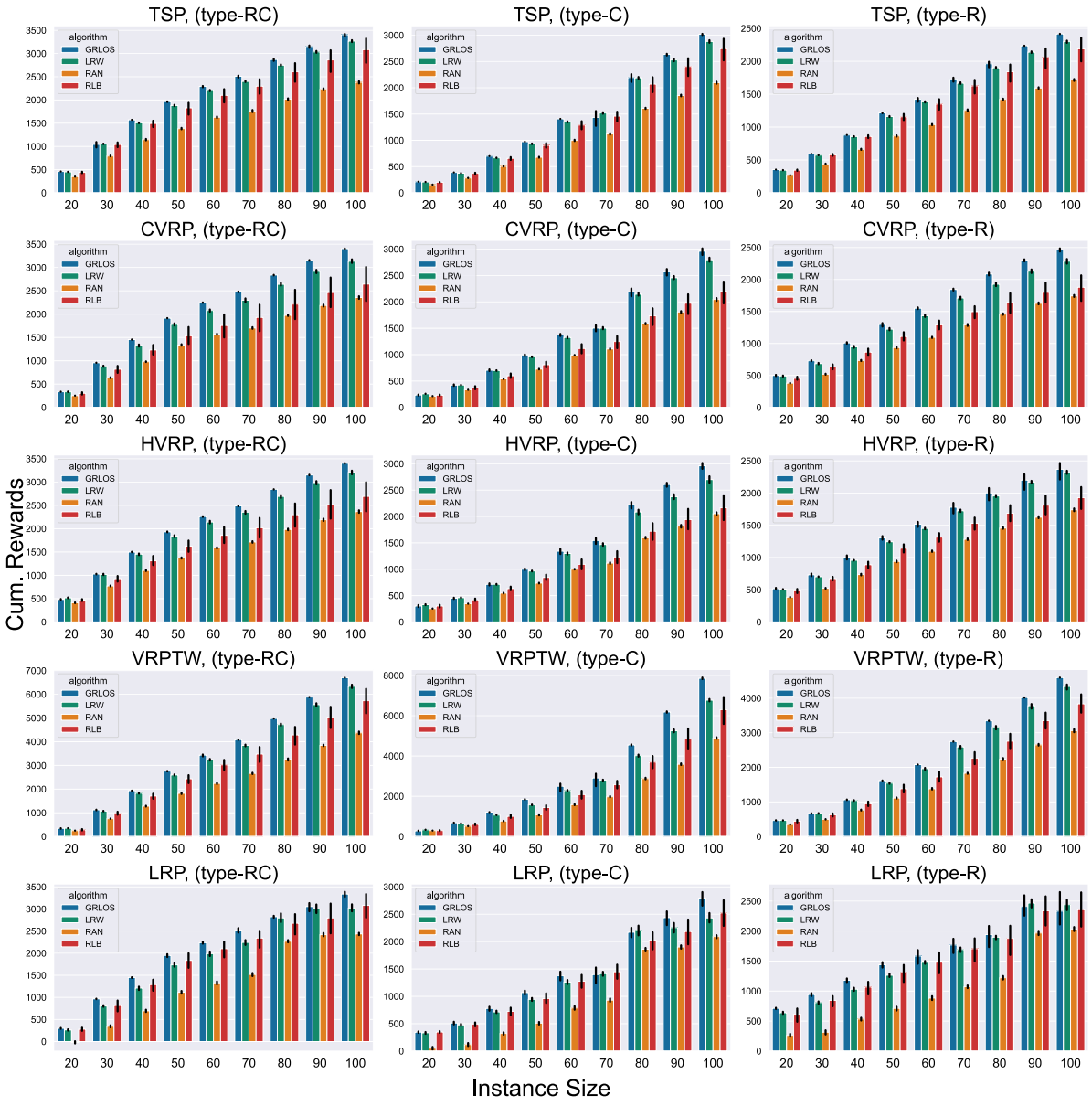


Fig. 2. Cumulative rewards (objective function improvements) obtained by the agents. Higher is better.

Table 4
Solution quality improvements made by each model over those obtained by RAN, averaged over Solomon dataset category and routing problem.

Model comparison	Dataset category			Routing problem				
	RC	C	R	TSP	CVRP	HVRP	VRPTW	LRP
GRLOS *versus* RAN	17.43%	13.21%	12.11%	17.86%	12.46%	10.94%	18.23%	9.41%
GRLOS-M *versus* RAN	21.31%	18.71%	16.64%	23.44%	18.55%	19.32%	22.98%	10.81%
LRW *versus* RAN	23.87%	19.39%	18.85%	**27.03%**	19.50%	20.16%	23.18%	13.61%
Hybrid *versus* RAN	**24.57%**	**20.31%**	**19.13%**	26.26%	**19.99%**	**20.40%**	**24.47%**	**14.53%**
CRW *versus* RAN	8.62%	7.21%	7.09%	10.52%	7.44%	7.94%	8.08%	4.36%
RLB *versus* RAN	15.64%	10.03%	7.92%	23.58%	6.11%	3.09%	11.14%	12.03%
RLB-M *versus* RAN	15.57%	9.79%	9.15%	23.29%	7.59%	5.80%	13.67%	7.16%

since no significant differences were observed across the R, C and RC instances in the random initial solution experiments (see, e.g., Fig. 2). We therefore restrict this set of experiments only to RC instances, since they feature both randomly placed and clustered customers, and are a composite of the C and R types.

We retain all settings in the experiments above, but instead generate initial solutions using the noisy Clarke-Wright algorithm as previously described. The results are presented in Fig. 4. Given a good-quality initial solution to start the search, the GRLOS and LRW models can still effectively accumulate positive rewards, showcasing that they are able to make improvements to a non-optimal initial solution. In contrast, the RAN baseline gains negative rewards, leading the search away and creating worse-quality incumbents. For the LRP variant, the only case in which RAN gains positive rewards, the scale of the improvement is considerably smaller compared to the GRLOS and LRW models.

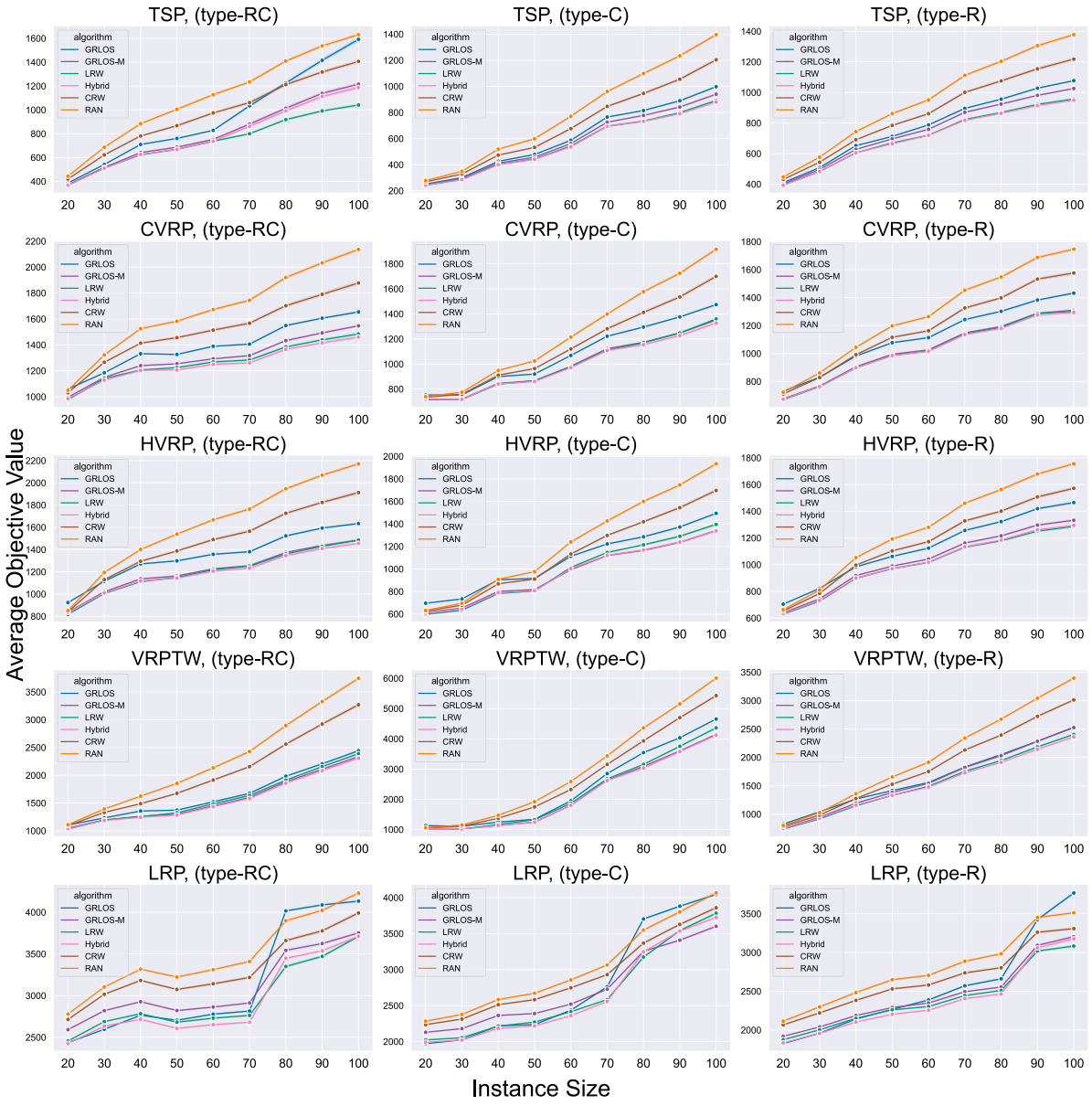


Fig. 3. Average objective function value during ALNS Evaluations for the different operator selection mechanisms on 5 routing problems and 3 Solomon dataset categories. Lower is better.

In Fig. 5, notably, the out-of-distribution degradation in GRLOS performance does not occur with these initial solutions, which can be explained by the fact that the solutions the model encounters are more similar to each other, as they are closer to optimality. Akin to the ALNS Evaluation with random initial solutions presented above, the reason for the sudden jump in objective values for LRP is the inclusion of an additional depot impacting the objective function values.

5.4. ALNS evaluation with varying ALNS operator budget

In this experiment, we carry out an ALNS Evaluation with a varying ALNS operator budget B_{ALNS}, so that we may analyse in detail how distribution shift affects the agents' performance. We fix the instance size to 100 and consider a value of B_{ALNS} ranging from 20 (equal to the training operator budget B_{MDP}) to 100. We observe from Fig. 6 that, given matching training and ALNS operator budgets, the GRLOS-based agents are able to more effectively reduce the objective value. Going beyond the training operator budget and hence out-of-distribution, the Hybrid and LRW agents begin to achieve better performance while standard GRLOS converges to no or poor solution improvements, which is partially mitigated by GRLOS-M. In other words, GRLOS can construct a good solution within similar training and ALNS operator budgets, but does not perform as well as the other agents for pure exploration. Once again, this reflects the limitation of ML models under distribution shift, which also applies to the RLB and RLB-M methods. Nonetheless, the advantage of GRLOS is its ability to efficiently find good solutions.

Certainly, training the model for the entire ALNS operator budget of 100 steps (i.e., setting $B_{\text{MDP}} = B_{\text{ALNS}} = 100$) is infeasible due to the number of possible sequences of operators ($\approx 17^{50} \times 7^{50}$ for the TSP and up to $25^{50} \times 7^{50}$ for the LRP variant). Hence, one necessarily needs to "cut off" the training operator budget at a certain point. Nevertheless, one may choose a larger budget, which would lead to an increase in performance for the ML approaches compared to the baselines, at the expense of requiring more training steps given the greater depth of the MDP.

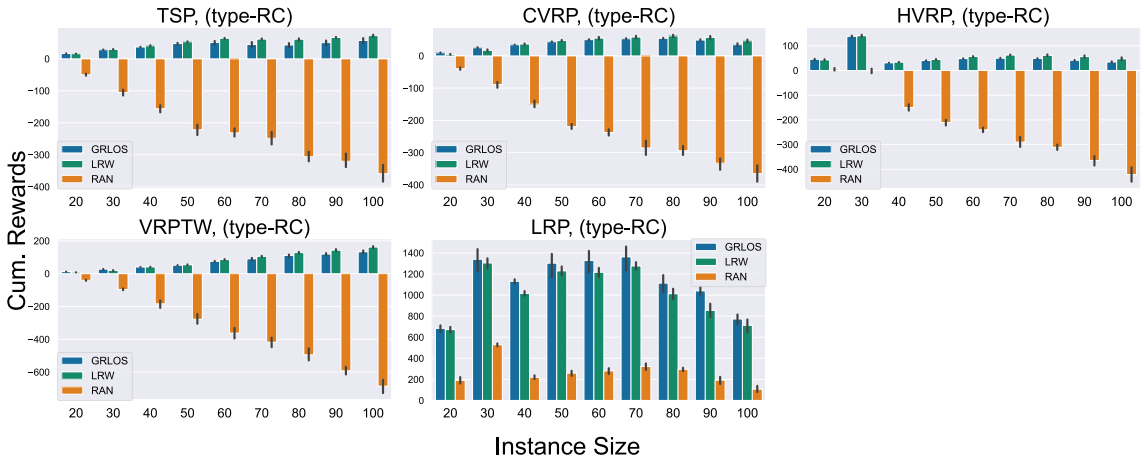


Fig. 4. Cumulative rewards obtained by the agents with noisy Clarke-Wright initial solutions. Higher is better.

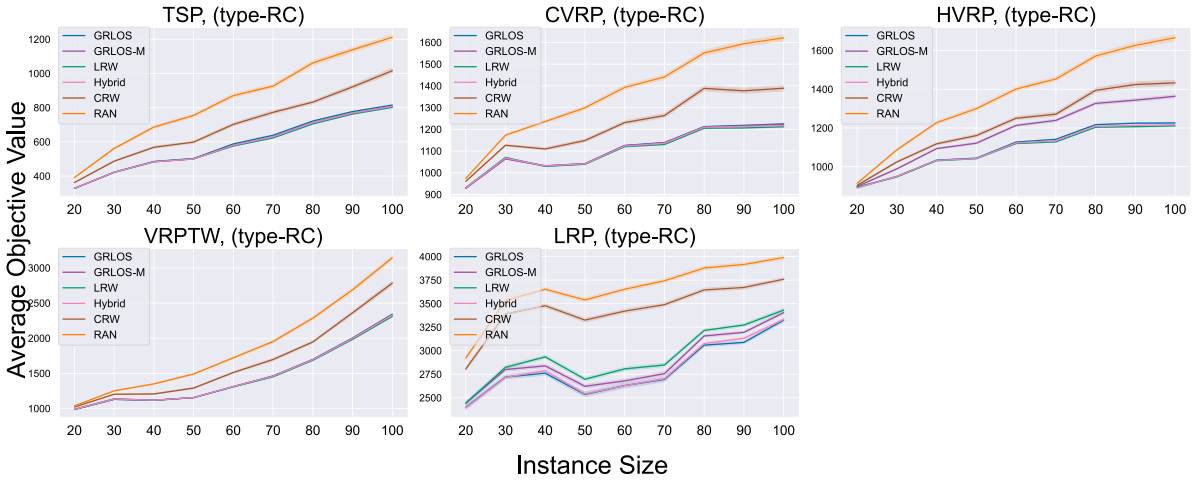


Fig. 5. Average objective function value during ALNS Evaluations for the different operator selection mechanisms on 5 routing problems and 3 Solomon dataset categories with noisy Clarke-Wright initial solutions. Lower is better.

Table 5
Average gap in objective value during ALNS Evaluations between GRLOS-M and RLB-M on 5 routing problems and 3 Solomon instance types. Positive values denote better comparative performance of GRLOS-M.

Variant	Instance	20	30	40	50	60	70	80	90	100
	RC	−0.08%	−1.08%	−0.38%	0.71%	1.69%	1.20%	0.78%	0.73%	0.28%
TSP	C	0.77%	1.42%	1.80%	2.15%	2.61%	0.51%	0.67%	1.33%	1.64%
	R	−0.32%	−0.57%	−1.18%	−0.45%	−1.01%	−1.02%	−1.00%	−0.47%	−0.54%
	RC	0.47%	2.08%	2.22%	6.80%	8.35%	8.55%	8.49%	9.39%	9.93%
CVRP	C	1.72%	4.87%	8.13%	12.44%	15.55%	15.72%	21.36%	23.53%	25.16%
	R	4.78%	8.03%	10.33%	13.70%	15.15%	17.83%	19.93%	21.06%	21.84%
	RC	0.75%	7.21%	9.89%	14.12%	15.82%	17.47%	18.01%	18.98%	19.53%
HVRP	C	2.14%	4.57%	9.87%	14.11%	17.55%	19.02%	25.06%	27.54%	29.31%
	R	−0.02%	2.22%	6.80%	10.82%	12.25%	14.15%	15.77%	16.92%	18.04%
	RC	1.70%	4.79%	7.90%	11.37%	12.82%	14.55%	14.59%	15.95%	16.39%
VRPTW	C	−0.48%	4.96%	12.82%	19.92%	15.60%	11.26%	16.01%	17.79%	18.57%
	R	0.55%	1.58%	4.20%	7.81%	10.15%	12.35%	13.59%	14.86%	15.52%
	RC	0.21%	1.79%	2.17%	2.02%	2.51%	3.43%	2.44%	3.53%	4.68%
LRP	C	1.37%	1.90%	1.97%	3.34%	3.82%	2.08%	3.01%	4.56%	4.51%
	R	4.08%	4.70%	5.65%	7.00%	6.96%	7.06%	7.93%	6.68%	5.85%

5.5. ALNS evaluation: Baseline comparison

In this experiment, we carry out an ALNS Evaluation and compare our GRLOS-M model to the RLB-M baseline, which adapts the state-of-the-art models proposed by Reijnen et al. (2024) and Kallestad et al. (2023) to our problem formulation and applies the out-of-distribution mitigation for fairness of comparison. The values in Table 5 are the percentage gaps in solution quality improvements, computed according to the formula $^{(\text{RLB-M}-\text{GRLOS-M})}/_{\text{RLB-M}} \times 100$.

We also carry out a Mann–Whitney U test with a 95% significance level. The few cells highlighted in grey represent tests whose outcome is not statistically significant, i.e. we cannot reject the null hypothesis that the performances of the two methods follow the same distribution. We observe that GRLOS-M outperforms the RLB-M baseline in 90.4% of cases in which there is a significant difference. Where RLB-M does

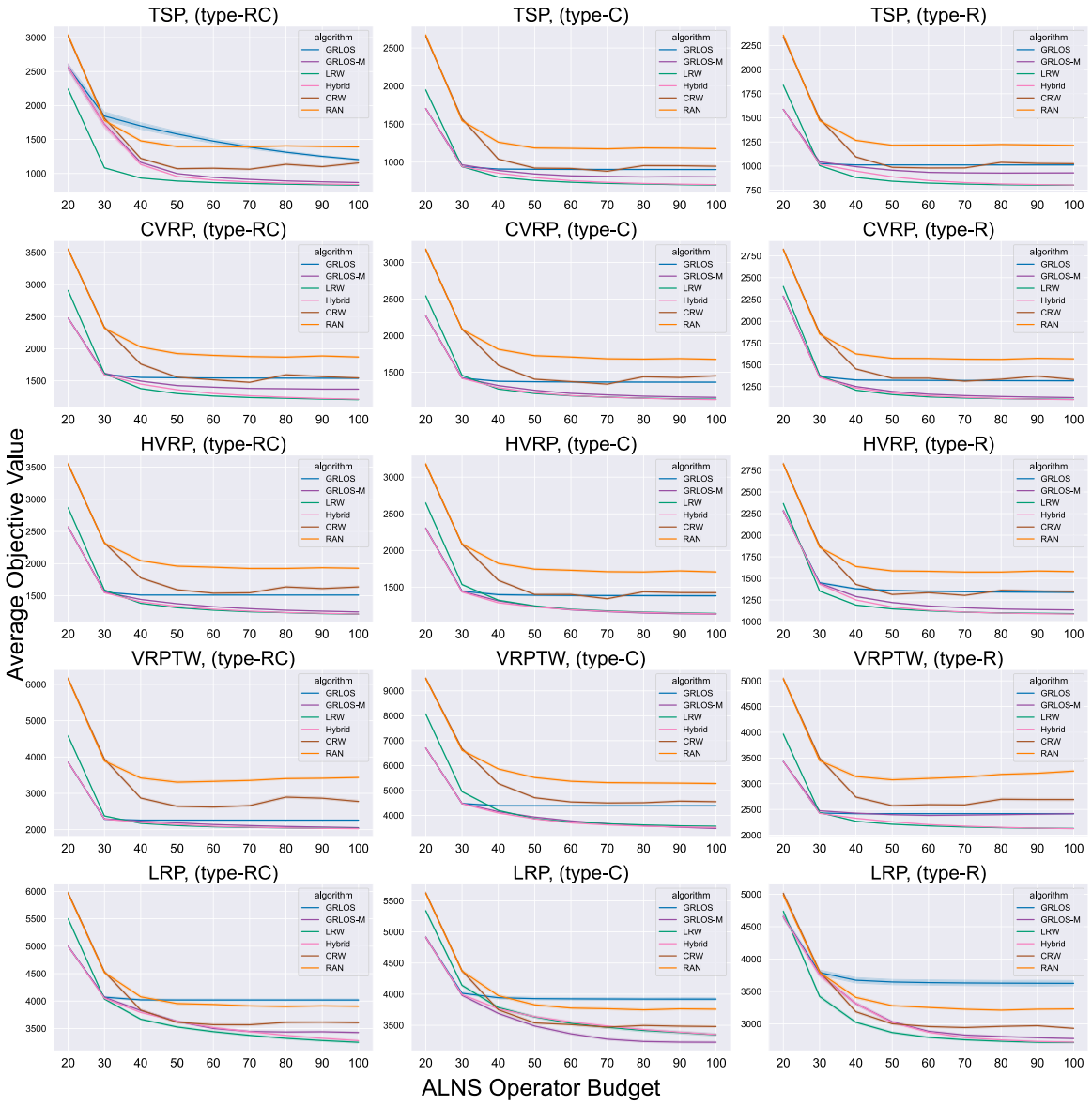


Fig. 6. Average objective function value during ALNS Evaluations as a function of operator budget (B_{ALNS}) for the different methods on 5 routing problems and 3 Solomon dataset categories. Lower is better.

perform better, the gap is less than a percentage point on average. As a general trend, our model excels with larger instances and problem variants with more intricate neighbourhood structures, emphasising the crucial role of operator selection.

5.6. Extrapolation to large instances

To further measure the extrapolation ability of the learning-based approaches, we evaluate the models trained on Solomon instances (with instance size available up to 100) on an unseen set of larger-scale GH instances (Gehring and Homberger, 1999) with up to 1000 nodes. We consider 128 sets of tours each for instance sizes in [200, 400, 600, 800, 1000]. As in the experiments presented in Section 5.3, we restrict this analysis to RC instances, as they display the characteristics of both R and C types and only small relative differences were found in the prior experiments. Out of the Deep RL methods, we only include the Hybrid method in this experiment as the results in Table 4 showed it performs best.

In Fig. 7, we observe that the average objective values of LRW are better than those of the RAN and CRW models, obtaining an average improvement of 31.09% and 22.24% respectively across all problem variants and instance sizes. This is consistent with the results on the Solomon dataset. Comparing Hybrid to RAN and CRW, we obtain improvements of 28.45% and 19.03%, respectively. We observe a slight degradation in performance of the Deep RL models compared to the Solomon dataset due to their limitations in out-of-distribution tasks: this setting, aside from different initial solutions, involves transferring the model to larger problem instances with different customer locations. The Hybrid model is nonetheless able to outperform the LRW model for HVRP and VRPTW, while obtaining similar performance to the LRW model for CVRP and LRP.

5.7. Computational time

The training process for a complete Solomon instance requires approximately 5 days using our reference pure Python implementation on a single core of a commodity CPU. Concretely, we utilised Intel Xeon E5-2637 v4 processors for reporting the measurements in this section. Most of the computational expense during the training process arises not from the model weight adjustment, but from the periodic validation

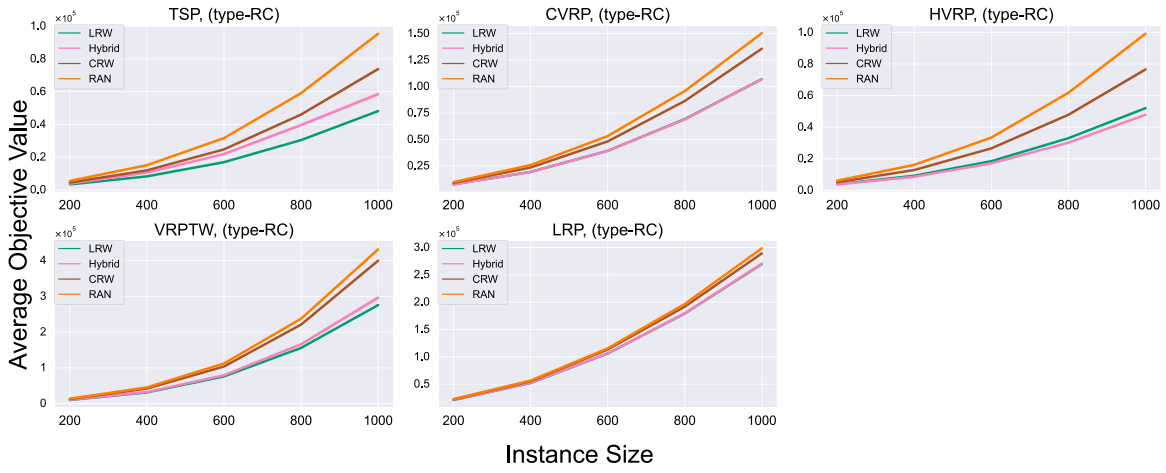


Fig. 7. Average objective function value during ALNS Evaluations for the different operator selection mechanisms on 5 routing problems and the large-scale Gehring and Homberger RC instance. Lower is better.

Table 6
The average computation time in second per individual ALNS Evaluation round with $B_{\text{ALNS}} = 50$ operators and 100 customers for different problem variants and Solomon instance types.

Variant	Type	GRLOS	GRLOS-M	LRW	Hybrid	RAN	CRW	RLB	RLB-M
TSP	RC	0.87	0.58	0.50	0.58	0.47	0.51	0.22	0.24
	C	0.85	0.63	0.50	0.63	0.53	0.56	0.29	0.31
	R	0.86	0.58	0.35	0.53	0.52	0.56	0.24	0.26
CVRP	RC	0.75	0.52	0.57	0.63	0.51	0.56	0.23	0.25
	C	0.61	0.48	0.56	0.55	0.50	0.55	0.37	0.48
	R	0.83	0.54	0.48	0.60	0.55	0.55	0.58	0.53
HVRP	RC	1.64	1.36	1.10	1.27	1.08	1.10	0.99	0.99
	C	1.77	1.35	1.12	1.38	1.01	1.11	1.56	1.28
	R	1.61	1.40	1.26	1.52	1.11	1.19	1.32	1.10
VRPTW	RC	3.18	2.85	2.39	2.72	2.23	2.43	2.77	2.25
	C	2.58	2.35	2.40	2.47	2.00	2.29	3.06	3.23
	R	1.97	1.78	2.57	2.55	2.22	2.41	3.18	2.39
LRP	RC	0.68	0.53	0.64	0.65	0.57	0.62	0.28	0.68
	C	0.69	0.48	0.60	0.64	0.56	0.59	0.67	1.14
	R	0.92	0.74	0.55	0.71	0.58	0.61	0.21	0.37

of the model, which requires applying the operators to all solutions in the validation set to assess performance. As such, the training cost can be reduced drastically with a more efficient implementation of the operator selection environment in a low-level programming language. Given training times are dominated by the validation costs, GRLOS, LRW, and RLB all require similar timeframes for training despite their different learning mechanisms.

The evaluation times within the ALNS framework are shown in Table 6. The computational time for a single iteration within the ALNS framework mostly takes less than 5 seconds. The maximum computational time per round is for VRPTW problems, partially due to the slightly longer time required for the specific destroy and repair operators. We observe that the GRLOS models are slightly more time-consuming than the others, yet they involve state-based information that helps to improve the training and evaluation performance at relatively small runtime costs. GRLOS-M computational times are strictly better than GRLOS since it performs a simple sampling instead of the neural network forward pass beyond the training horizon.

6. Conclusion and future direction

In this work, we have addressed the problem of effective operator selection in the ALNS metaheuristic. We have isolated the problem and formulated it as a Markov Decision Process, in which an agent is rewarded proportionally to the improvement in the objective function value of the solution. We proposed two mechanisms for learning operator selection policies: GRLOS, which combines an Reinforcement Learning technique with Graph Neural Network representations of the state, and LRW, a simple learning-based technique inspired by the classic Roulette Wheel update. The two approaches are complementary: GRLOS is conditioned on the decision space characteristics of the current solution and more powerful within a horizon similar to its training conditions, while the LRW is independent of the current solution and more lightweight as a result.

The following insights have emerged from our evaluation. GRLOS and LRW both consistently outperform the classic Roulette Wheel update and uniform random operator selection. GRLOS is able to learn strategies that are statistically significantly more performant than those produced by LRW when the distribution of solutions for evaluation matches that seen during training, but may suffer in performance under distribution shifts. A hybrid approach combining GRLOS and LRW also yields better performance, on aggregate, than either method alone. GRLOS is shown to outperform a method based on concurrent works that use RL (Kallestad et al., 2023; Reijnen et al., 2024) in 90.4% of cases. Furthermore, the findings extend to initial solutions generated using the Clarke-Wright heuristic, and successfully generalise to much larger problem instances with up to 1000 nodes in the case of the LRW and Hybrid methods.

We would like to highlight the following directions for future work. Applications to other combinatorial optimisation problems for which ALNS is amenable can be considered. This requires (1) the specification of action spaces D and R corresponding to problem-relevant destroy and repair operators, which may be designed or learned; and (2) defining the node and edge feature matrices $\mathbf{X}^{\text{node}}, \mathbf{X}^{\text{edge}}$. Herein, as

performed in the classic ALNS framework, we have considered a predefined destroy scale for the operators. Further work may consider incorporating the choice of destroy scale into the decision-making process for the RL agent, which may yield finer-grained control at the expense of increases in the breadth and depth of the MDP. Finally, the robustness of RL approaches when applied to states different from those seen during training remains an open question, but an important one to address if hybrid approaches combining metaheuristics and Machine Learning are to be successful.

CRediT authorship contribution statement

Syu-Ning Johnn: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Formal analysis, Conceptualization. **Victor-Alexandru Darvariu:** Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation, Formal analysis. **Julia Handl:** Supervision. **Jörg Kalcsics:** Supervision.

Data availability

We have used two publicly available datasets for our experiments and have cited the sources in the manuscript. Processed versions of these datasets are made available together with the released code.

Acknowledgements

This work was supported by the University of Edinburgh School of Mathematics and partially supported by The Alan Turing Institute under the Enrichment Scheme and the UK EPSRC grant EP/N510129/1. Experiments for this work were carried out on the UCL Department of Computer Science High Performance Computing (HPC) cluster.

Appendix A. Code and data

Code and data are publicly available at https://github.com/SYU-NING/grl-alns-framework, so that others may reproduce and build on our work. This includes instructions for setting up the code and running the experiments to enable reproducing all reported results.

Appendix B. ALNS operators literature review

The complete list of considered destroy and repair operators, together with descriptions of their workings and references to the relevant papers, is given in Table B.7.

Table B.7
ALNS destroy and repair operators summary. * means the operator mechanism has been adjusted to fit specific neighbourhood type (e.g., time to demand-based) without changing the mechanism itself. q is denoted as the destroy/repair scale.

Operator type	Detailed operator	Description	Reference
Random-based Destroy Operators	Random Node Destroy	Removes a set of q randomly-selected nodes from their existing routes and placed inside the customer pool.	Ropke and Pisinger (2006)
	Random Route Destroy	Randomly and iteratively selects a route from the solution and remove a set of q nodes sequentially from the route to the customer pool based on the visiting sequence, repeat this process until the destroy budget is met.	Demir et al. (2012)
	Forbidden Random Destroy	Randomly removes a set of q nodes that have not been removed from the solution for a minimum number of times in the preceding iterations.	Li et al. (2016)
	Close-an-Open-Facility Destroy	Randomly closes an open facility and removes all its allocated nodes from the solution to the customer pool. Bypass the destroy budget requirement if the number of allocated customers exceeds the destroy scale.	Hemmelmayr et al. (2012)
	Open-a-Closed-Facility Destroy	Randomly opens a closed facility and removes a set of q nodes from the existing routes that are located in the immediate geographical vicinity of this facility and places them inside the customer pool.	Hemmelmayr et al. (2012)
Greedy-based Destroy Operators	Worst Distance Destroy	Removes a set of q nodes with the highest removal gains computed from the geographical distance to their successive nodes.	Ropke and Pisinger (2006)
	Route Deviation Destroy	Removes a set of q nodes from the routes which total distances exhibit the highest deviation from the average route length.	Demir et al. (2012)
	Greedy Route Longest Destroy	Removes a set of q nodes from the worst routes with the longest total travelling distance.	Demir et al. (2012)*
	Greedy Route Shortest Destroy	Removes a set of q nodes from the worst routes with the shortest total travelling distance.	Demir et al. (2012)*
	Historical Knowledge Destroy	Removes a set of q nodes with the highest travel distance discrepancies from their successive nodes based on the historical record and place them inside the customer pool.	Alinaghian and Shokouhi (2018)
	Largest Demand Deviation Destroy	Removes a set of q nodes with their demands exhibiting the highest deviation from the average.	Demir et al. (2012)*
	Smallest Demand Destroy	Removes a set of q nodes with the smallest demands from the population.	Demir et al. (2012)
	Largest Demand Destroy	Removes a set of q nodes with the highest demands from the population.	Demir et al. (2012)
	Worst Facility Node Destroy	Randomly selects a facility and removes a set of q nodes with the largest removal gains until the destroy budget is met.	Hemmelmayr et al. (2012)*
	Worst Time Destroy	Removes a set of q nodes with the most significant deviations in service starting time from the average estimate and places them inside the customer pool.	Keskin and Çatay (2016)*
Related-based Destroy Operators	Proximity Distance Destroy	Randomly selects a single node and removes it together with a set of $q-1$ nodes that are closely associated in terms of the highest geographical proximity.	Lehuédé et al. (2014)
	Pair Destroy	Randomly selects a set of $q/2$ nodes to place inside the customer pool, then for each node removes its geographically nearest paired acquaintance that has not yet been removed from the route.	Mancini (2016)
	Cluster Destroy	Ranks pairs of nodes according to their inter-travelling distance in ascending order and sequentially removes nodes with the smallest distance that still persist within the solution. This algorithm is based on the Kruskal's algorithm for generating minimum spanning trees, to separate two or multiple groups of nodes that are geographically far away from each other.	Pisinger and Ropke (2007)

(*continued on next page*)

Table B.7 (*continued*).

Operator type	Detailed operator	Description	Reference
	Node Neighbourhood Destroy	Randomly selects a single node and removes it together with a set of $q-1$ nodes that lie within a rectangular boundary surrounding the selected node.	Alinaghian and Shokouhi (2018)
	Zone Destroy	Randomly selects a single node and removes it together with $q-1$ nodes within a predefined region using the Cartesian coordinate system encompassing the node.	Emeç et al. (2016)
	Route Neighbourhood Destroy	Randomly chooses a pair of routes and iteratively removes a pair of customers from each route with the shortest inter-node travelling distance until the destroy budget is met.	Emeç et al. (2016)
	Historical Pair Destroy	Compute the number of times the any two nodes x and y have been served by the same vehicle. This is used as the related weight in Shaw removal.	Tellez et al. (2018)
	Shaw Destroy	Removes a random single node and the remaining $q-1$ nodes with the highest relatedness measures to the single node and places inside the customer pool. Relatedness is computed using a weighted sum of $\alpha \times$ geographical distance $+ \gamma \times$ same vehicle status $+ \delta \times$ demand difference, with an additional term $\beta \times$ start time difference if VRPTW variant is applied.	Ropke and Pisinger (2006)
	Proximity Demand Destroy	Removes a random single node and a set of $q-1$ nodes with the most similar demand level and places inside the customer pool.	Demir et al. (2012)
	Facility Swap Destroy	Exchanges the statuses of a pair of open and closed facilities with a probability inversely proportional to their geographical travelling distance.	Hemmelmayr et al. (2012)
	Proximity Time Destroy	Removes a random single node and a set of $q-1$ nodes that have the most similar service starting times and places inside the customer pool.	Demir et al. (2012)
	Proximity Service Destroy	Removes a random single node and a set of $q-1$ nodes that have the most similar service lengths and places inside the customer pool.	Pisinger and Ropke (2007)
	Do Nothing Operator	Make no change to the current graph simply pass on the current solution to the next state.	(Our work)
Repair Operators	Random Repair	Randomly selects a node and insert it in a random position during each iteration until the customer pool is empty.	Qu and Bard (2012)
	Greedy Repair	During each iteration, randomly picks a node from the customer pool and place it in the position that minimally increases the total costs. The insertion can be between two consecutive customer nodes or between the facility and its linking customer node.	Ropke and Pisinger (2006)
	Greedy Perturbation Repair	Incorporates the identical mechanism as previously described under Greedy Repair. The insertion cost at each specific position is influenced by a perturbation factor d randomly drawn between [0.8, 1.2].	Contardo et al. (2012)
	Deep Greedy Repair	Ranks the nodes inside the customer pool from the lowest to highest using the insertion cost, then insert the nodes in accordance with the specified order using Greedy Repair.	Laporte et al. (2010)
	Regret-2 Repair	Ranks the nodes inside customer pool from the highest to lowest using the regret value, which is computed as the insertion cost difference for the first and kth cheapest position, where k = 2.	Ribeiro and Laporte (2012)
	Regret-3 Repair	Same as above with k = 3.	Ribeiro and Laporte (2012)
	Regret-4 Repair	Same as above with k = 4.	Ribeiro and Laporte (2012)

References

Ahmadi, E., Goldengorin, B., Süer, G.A., Mosadegh, H., 2018. A hybrid method of 2-TSP and novel learning-based GA for job sequencing and tool switching problem. Appl. Soft Comput. 65, 214–229.

Ahn, S., Seo, Y., Shin, J., 2020. Learning what to defer for maximum independent sets. In: ICML.

Alinaghian, M., Shokouhi, N., 2018. Multi-depot multi-compartment vehicle routing problem, solved by a hybrid adaptive large neighborhood search. Omega 76, 85–99.

Bai, R., Chen, X., et al., 2023. Analytics and machine learning in vehicle routing research. Int. J. Prod. Res. 61 (1), 4–30.

Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S., 2016. Neural combinatorial optimization with reinforcement learning. In: ICLR Workshops.

Bengio, Y., Lodi, A., Prouvost, A., 2021. Machine learning for combinatorial optimization: a methodological tour d'horizon. European J. Oper. Res. 290 (2), 405–421.

Bongiovanni, C., Kaspi, M., Cordeau, J.-F., Geroliminis, N., 2022. A machine learning-driven two-phase metaheuristic for autonomous ridesharing operations. Transp. Res. E: Logist. Transp. Rev. 165, 102835.

Burke, E.K., Gendreau, M., Hyde, M., Kendall, G., Ochoa, G., Özcan, E., Qu, R., 2013. Hyper-heuristics: A survey of the state of the art. J. Oper. Res. Soc. 64, 1695–1724.

Chen, X., Tian, Y., 2019. Learning to perform local rewriting for combinatorial optimization. In: NeurIPS.

Chi, C., Aboussalah, A., Khalil, E., Wang, J., Sherkat-Masoumi, Z., 2022. A deep reinforcement learning framework for column generation. In: NeurIPS.

Chua, K., Calandra, R., McAllister, R., Levine, S., 2018. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In: NeurIPS.

Clarke, G., Wright, J.W., 1964. Scheduling of vehicles from a central depot to a number of delivery points. Oper. Res. 12 (4), 568–581.

Contardo, C., Hemmelmayr, V., Crainic, T.G., 2012. Lower and upper bounds for the two-echelon capacitated location-routing problem. Comput. Oper. Res. 39 (12), 3185–3199.

Cybenko, G., 1989. Approximation by superpositions of a sigmoidal function. Math. Control Signals Syst. 2 (4), 303–314.

Dai, H., Dai, B., Song, L., 2016. Discriminative embeddings of latent variable models for structured data. In: ICML.

Darvariu, V.-A., Hailes, S., Musolesi, M., 2021. Goal-directed graph construction using reinforcement learning. Proc. R. Soc. A 477 (2254), 20210168.

Demir, E., Bektaş, T., Laporte, G., 2012. An adaptive large neighborhood search heuristic for the pollution-routing problem. European J. Oper. Res. 223 (2), 346–359.

Deudon, M., Cournut, P., Lacoste, A., Adulyasak, Y., Rousseau, L.-M., 2018. Learning heuristics for the TSP by policy gradient. In: CPAIOR.

Durgut, R., Aydin, M.E., Atli, I., 2021. Adaptive operator selection with reinforcement learning. Inform. Sci. 581, 773–790.

Emeç, U., Çatay, B., Bozkaya, B., 2016. An adaptive large neighborhood search for an e-grocery delivery routing problem. Comput. Oper. Res. 69, 109–125.

Falkner, J.K., Thyssens, D., Schmidt-Thieme, L., 2022. Large neighborhood search based on neural construction heuristics. arXiv:2205.00772.

Gehring, H., Homberger, J., 1999. A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows. In: EUROGEN99.

Grangier, P., Gendreau, M., Lehuédé, F., Rousseau, L.-M., 2017. A matheuristic based on large neighborhood search for the vehicle routing problem with cross-docking. Comput. Oper. Res. 84, 116–126.

Hamilton, W.L., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. In: NeurIPS.

Handoko, S.D., Nguyen, D.T., Yuan, Z., Lau, H.C., 2014. Reinforcement learning for adaptive operator selection in memetic search applied to quadratic assignment problem. In: GECCO Companion.

Hemmelmayr, V.C., Cordeau, J.-F., Crainic, T.G., 2012. An adaptive large neighborhood search heuristic for two-echelon vehicle routing problems arising in city logistics. Comput. Oper. Res. 39 (12), 3215–3228.

Hopfield, J.J., Tank, D.W., 1985. "Neural" computation of decisions in optimization problems. Biol. Cybernet. 52 (3), 141–152.

Hottung, A., Tierney, K., 2020. Neural large neighborhood search for the capacitated vehicle routing problem. In: ECAI.

Johnn, S.-N., Darvariu, V.-A., Handl, J., Kalcsics, J., 2023. Graph reinforcement learning for operator selection in the ALNS metaheuristic. In: International Conference on Optimization and Learning. Springer, pp. 200–212.

Kalatzantonakis, P., Sifaleras, A., Samaras, N., 2023. A reinforcement learning-variable neighborhood search method for the capacitated vehicle routing problem. Expert Syst. Appl. 213, 118812.
Kallestad, J., Hasibi, R., Hemmati, A., Sörensen, K., 2023. A general deep reinforcement learning hyperheuristic framework for solving combinatorial optimization problems. European J. Oper. Res. 309 (1), 446–468.
Karimi-Mamaghan, M., Mohammadi, M., Meyer, P., Karimi-Mamaghan, A.M., Talbi, E.-G., 2022. Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art. European J. Oper. Res. 296 (2), 393–422.
Karimi-Mamaghan, M., Mohammadi, M., Pasdeloup, B., Meyer, P., 2023. Learning to select operators in meta-heuristics: An integration of Q-learning into the iterated greedy algorithm for the permutation flowshop scheduling problem. European J. Oper. Res. 304 (3), 1296–1330.
Keskin, M., Çatay, B., 2016. Partial recharge strategies for the electric vehicle routing problem with time windows. Transp. Res. C: Emerg. Technol. 65, 111–127.
Khalil, E., Dai, H., Zhang, Y., Dilkina, B., Song, L., 2017. Learning combinatorial optimization algorithms over graphs. In: NeurIPS.
Kim, M., Park, J., et al., 2021. Learning collaborative policies to solve NP-hard routing problems. In: NeurIPS.
Kool, W., Van Hoof, H., Welling, M., 2018. Attention, learn to solve routing problems!. In: ICLR.
Lagos, F., Pereira, J., 2024. Multi-armed bandit-based hyper-heuristics for combinatorial optimization problems. European J. Oper. Res. 312 (1), 70–91.
Laporte, G., Musmanno, R., Vocaturo, F., 2010. An adaptive large neighbourhood search heuristic for the capacitated arc-routing problem with stochastic demands. Transp. Sci. 44 (1), 125–135.
Lehuédé, F., Masson, R., Parragh, S.N., Péton, O., Tricoire, F., 2014. A multi-criteria large neighbourhood search for the transportation of disabled people. J. Oper. Res. Soc. 65 (7), 983–1000.
Li, B., Krushinsky, D., Van Woensel, T., Reijers, H.A., 2016. An adaptive large neighborhood search heuristic for the share-a-ride problem. Comput. Oper. Res. 66, 170–180.
López-Ibáñez, M., Dubois-Lacoste, J., Cáceres, L.P., Birattari, M., Stützle, T., 2016. The irace package: Iterated racing for automatic algorithm configuration. Oper. Res. Perspect. 3, 43–58.
Lu, H., Zhang, X., Yang, S., 2019. A learning-based iterative method for solving vehicle routing problems. In: ICLR.
Mancini, S., 2016. A real-life multi depot multi period vehicle routing problem with a heterogeneous fleet: Formulation and adaptive large neighborhood search based matheuristic. Transp. Res. C: Emerg. Technol. 70, 100–112.
Mara, S.T.W., Norcahyo, R., Jodiawan, P., Lusiantoro, L., Rifai, A.P., 2022. A survey of adaptive large neighborhood search algorithms and applications. Comput. Oper. Res. 146, 105903.
Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., et al., 2015. Human-level control through deep reinforcement learning. Nature 518 (7540), 529–533.
Mosadegh, H., Ghomi, S.F., Süer, G.A., 2020. Stochastic mixed-model assembly line sequencing problem: Mathematical modeling and Q-learning based simulated annealing hyper-heuristics. European J. Oper. Res. 282 (2), 530–544.
Nair, V., Bartunov, S., Gimeno, F., von Glehn, I., Lichocki, P., Lobov, I., O'Donoghue, B., Sonnerat, N., Tjandraatmadja, C., Wang, P., et al., 2020. Solving mixed integer programs using neural networks. arXiv:2012.13349.
Nazari, M., Oroojlooy, A., Snyder, L., Takác, M., 2018. Reinforcement learning for solving the vehicle routing problem. In: NeurIPS.
Oberweger, F.F., Raidl, G.R., Rönnberg, E., Huber, M., 2022. A learning large neighborhood search for the staff rerostering problem. In: CPAIOR.
Pisinger, D., Ropke, S., 2007. A general heuristic for vehicle routing problems. Comput. Oper. Res. 34 (8), 2403–2435.
Qiu, H., Wang, S., Yin, Y., Wang, D., Wang, Y., 2022. A deep reinforcement learning-based approach for the home delivery and installation routing problem. Int. J. Prod. Econ. 244, 108362.
Qu, Y., Bard, J.F., 2012. A GRASP with adaptive large neighborhood search for pickup and delivery problems with transshipment. Comput. Oper. Res. 39 (10), 2439–2456.
Quiñonero-Candela, J., Sugiyama, M., Schwaighofer, A., Lawrence, N.D., 2008. Dataset Shift in Machine Learning. MIT Press.
Reijnen, R., Zhang, Y., Lau, H.C., Bukhsh, Z., 2024. Operator selection in adaptive large neighborhood search using deep reinforcement learning. In: CPAIOR.
Ribeiro, G.M., Laporte, G., 2012. An adaptive large neighborhood search heuristic for the cumulative capacitated vehicle routing problem. Comput. Oper. Res. 39 (3), 728–735.
Ropke, S., Pisinger, D., 2006. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transp. Sci. 40 (4), 455–472.
Santini, A., Ropke, S., Hvattum, L.M., 2018. A comparison of acceptance criteria for the adaptive large neighbourhood search metaheuristic. J. Heuristics 24 (5), 783–815.
Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M., Monfardini, G., 2009. The graph neural network model. IEEE Trans. Neural Netw. 20 (1), 61–80.
Shaw, P., 1999. Using constraint programming and local search methods to solve vehicle routing problems. In: Principles and Practice of Constraint Programming — CP98.
Smith, K.A., 1999. Neural networks for combinatorial optimization: A review of more than a decade of research. INFORMS J. Comput. 11 (1), 15–34.
Solomon, M.M., 1987. Algorithms for the vehicle routing and scheduling problems with time window constraints. Oper. Res. 35 (2), 254–265.
Song, J., Yue, Y., Dilkina, B., et al., 2020. A general large neighborhood search framework for solving integer linear programs. In: NeurIPS.
Sperduti, A., Starita, A., 1997. Supervised neural networks for the classification of structures. IEEE Trans. Neural Netw. 8 (3), 714–735.
Syed, A.A., Akhnoukh, K., Kaltenhaeuser, B., Bogenberger, K., 2019. Neural network based large neighborhood search algorithm for ride hailing services. In: EPIA Conference on Artificial Intelligence.
Talbi, E.-G., 2021. Machine learning into metaheuristics: A survey and taxonomy. ACM Comput. Surv. 54 (6), 1–32.
Tellez, O., Vercraene, S., Lehuédé, F., Péton, O., Monteiro, T., 2018. The fleet size and mix dial-a-ride problem with reconfigurable vehicle capacity. Transp. Res. C: Emerg. Technol. 91, 99–123.
Thomas, C., Schaus, P., 2018. Revisiting the self-adaptive large neighborhood search. In: CPAIOR.
Turkeš, R., Sörensen, K., Hvattum, L.M., 2021. Meta-analysis of metaheuristics: Quantifying the effect of adaptiveness in adaptive large neighborhood search. European J. Oper. Res. 292 (2), 423–442.
Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., 2018. Graph attention networks. In: ICLR.
Watkins, C., Dayan, P., 1992. Q-learning. Mach. Learn. 8 (3–4), 279–292.
Wu, Y., Song, W., Cao, Z., Zhang, J., 2021a. Learning large neighborhood search policy for integer programming. In: NeurIPS.
Wu, Y., Song, W., Cao, Z., Zhang, J., Lim, A., 2021b. Learning improvement heuristics for solving routing problems. IEEE Trans. Neural Netw. Learn. Syst. 33 (9), 5057–5069.
You, J., Liu, B., Ying, R., Pande, V., Leskovec, J., 2018. Graph convolutional policy network for goal-directed molecular graph generation. In: NeurIPS.
Zhang, Y., Bai, R., Qu, R., Tu, C., Jin, J., 2022. A deep reinforcement learning based hyper-heuristic for combinatorial optimisation with uncertainties. European J. Oper. Res. 300 (2), 418–427.
Zhang, K., He, F., Zhang, Z., Lin, X., Li, M., 2020. Multi-vehicle routing problems with soft time windows: A multi-agent reinforcement learning approach. Transp. Res. C: Emerg. Technol. 121, 102861.
Zhao, F., Zhang, L., Cao, J., Tang, J., 2021. A cooperative water wave optimization algorithm with reinforcement learning for the distributed assembly no-idle flowshop scheduling problem. Comput. Ind. Eng. 153, 107082.
Zhou, Y., Hao, J.-K., Duval, B., 2016. Reinforcement learning based local search for grouping problems: A case study on graph coloring. Expert Syst. Appl. 64, 412–422.